

(51) **Int. Cl.**

G06T 3/4015 (2024.01)

G06T 5/20 (2006.01)

(58) **Field of Classification Search**

CPC . G06T 2207/10021; G06T 2207/20032; G06T
2207/20084; G06T 3/40; G06T 3/4038;
G06T 3/4053; G06T 7/50; G06T 7/70;
G02B 5/20; G06N 3/045; G06N 3/08

See application file for complete search history.

(56)

References Cited

U.S. PATENT DOCUMENTS

2013/0076939	A1 *	3/2013	Kaizu	H04N 23/843 348/224.1
2017/0078596	A1	3/2017	Chesnokov et al.	
2017/0318300	A1 *	11/2017	Suzuki	H04N 19/186
2019/0139189	A1	5/2019	Srinivasamurthy et al.	
2019/0318451	A1	10/2019	Swami et al.	
2019/0379812	A1 *	12/2019	Suman	H04N 23/45
2020/0051210	A1 *	2/2020	Wang	G06T 3/4015
2021/0192685	A1 *	6/2021	Nomura	H04N 23/843
2021/0243352	A1	8/2021	McElvain et al.	
2022/0207680	A1 *	6/2022	Wang	G06T 5/73

* cited by examiner

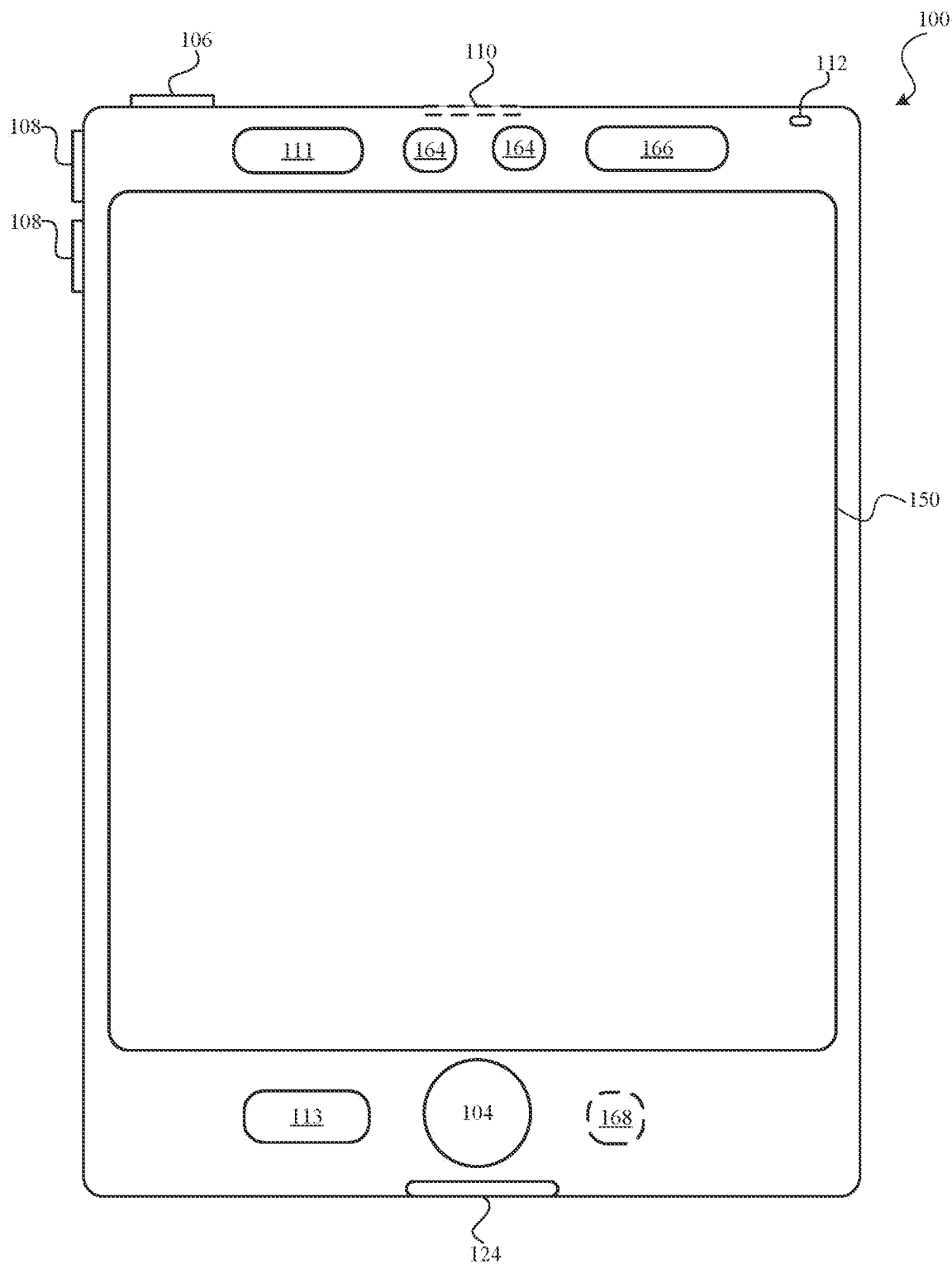


FIG. 1

100

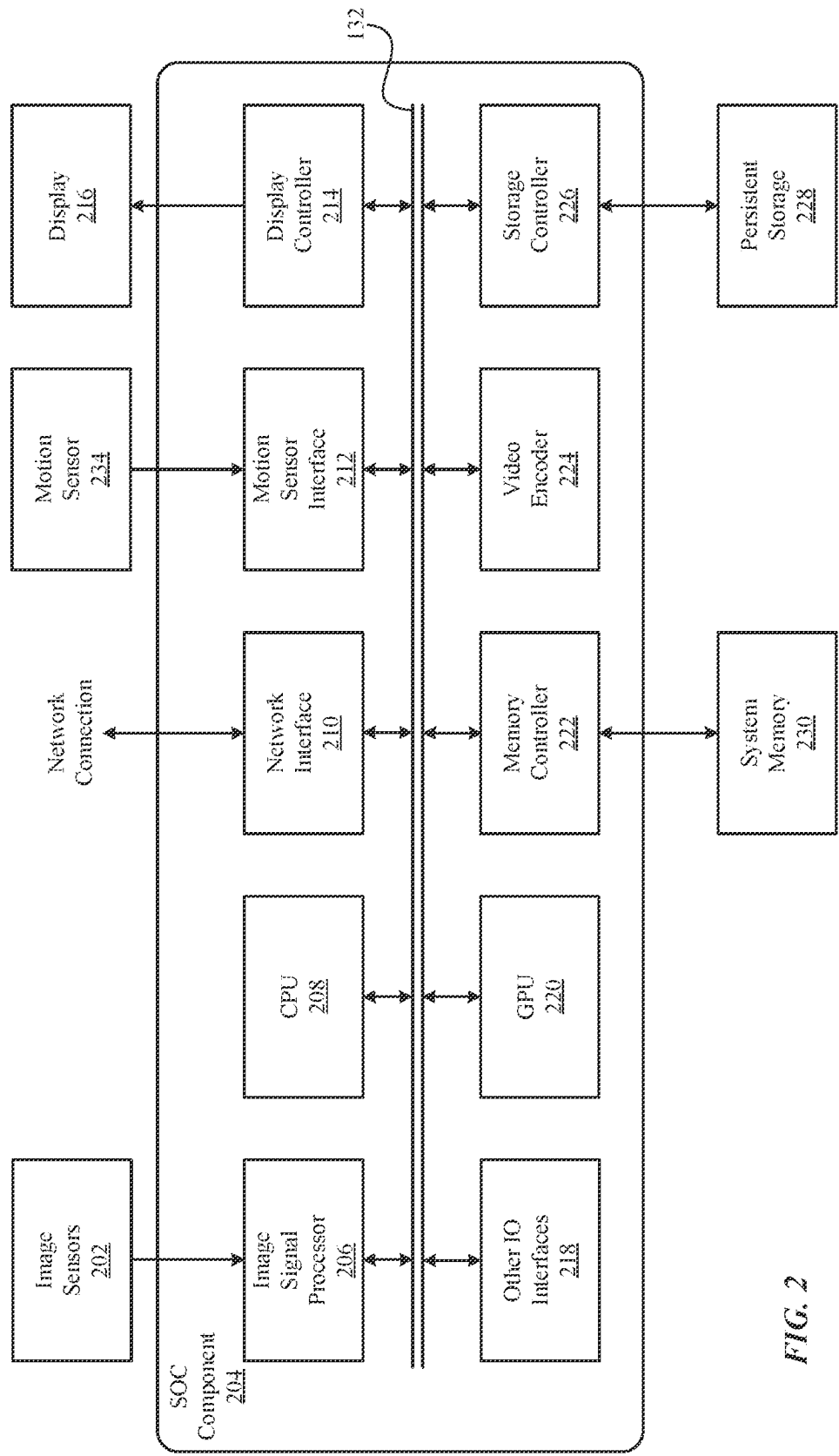


FIG. 2

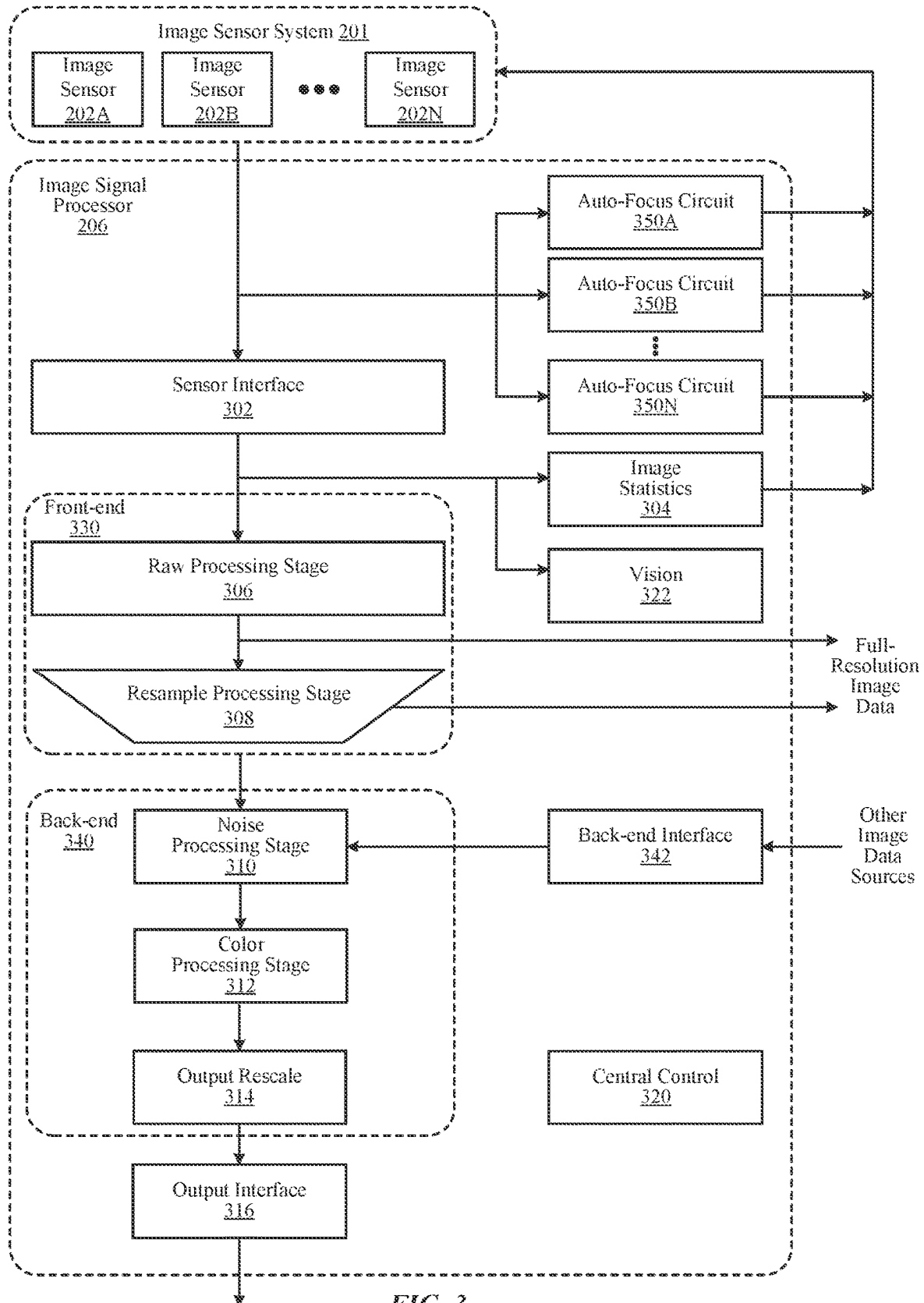


FIG. 3

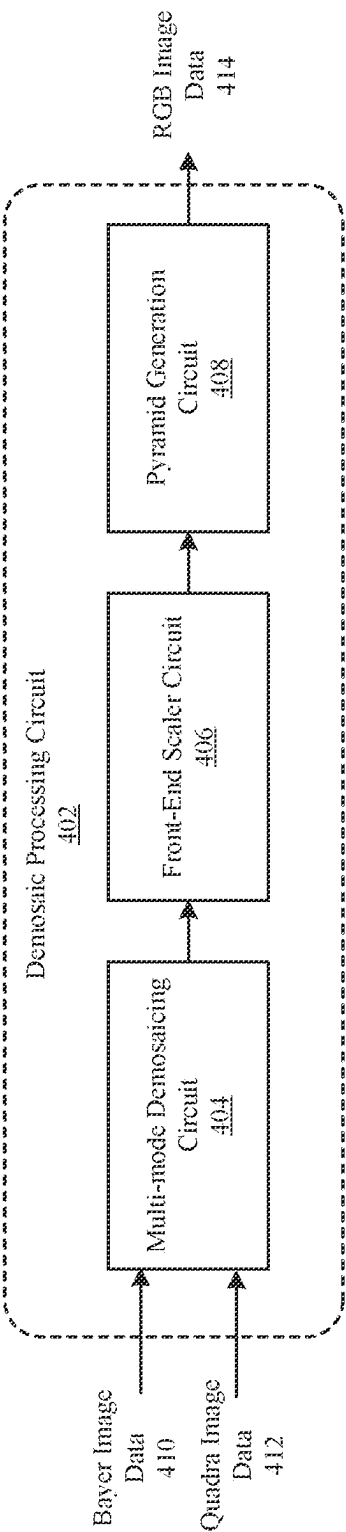


FIG. 4

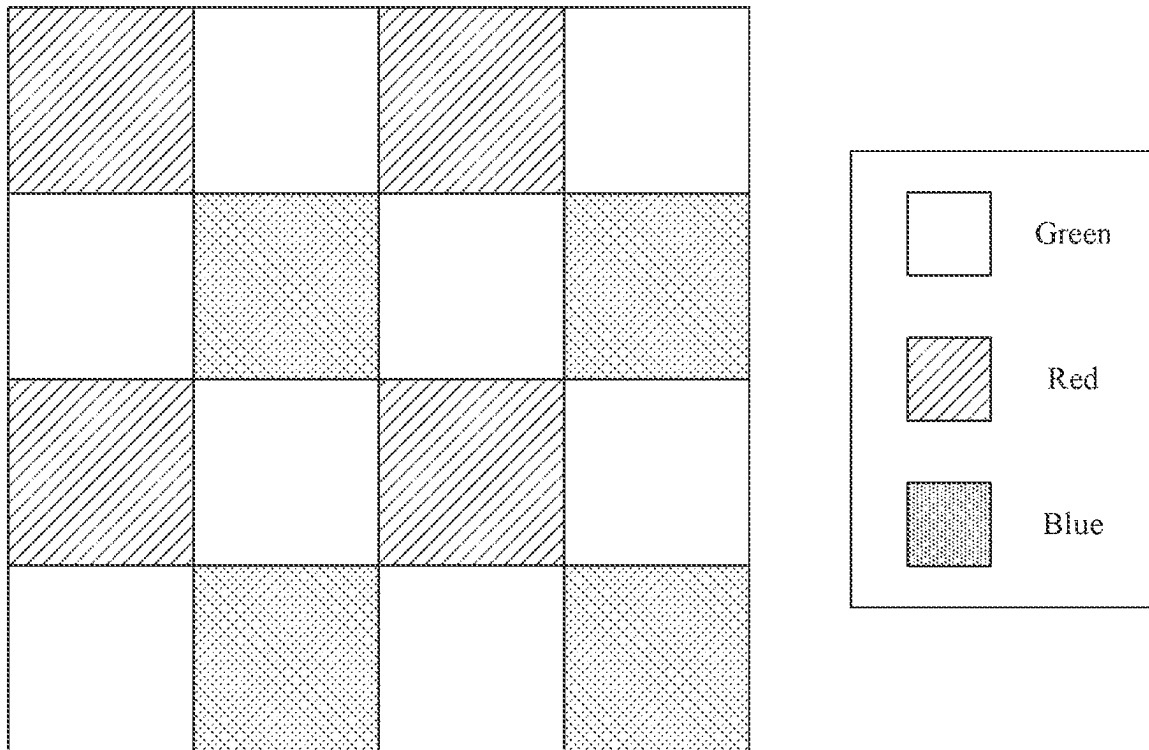


FIG. 5A

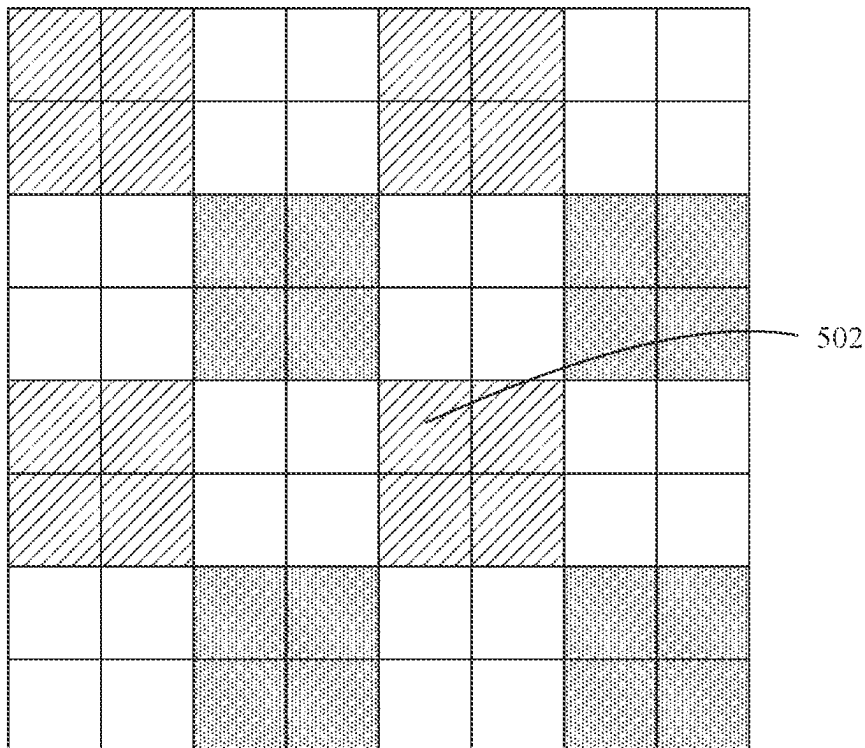


FIG. 5B

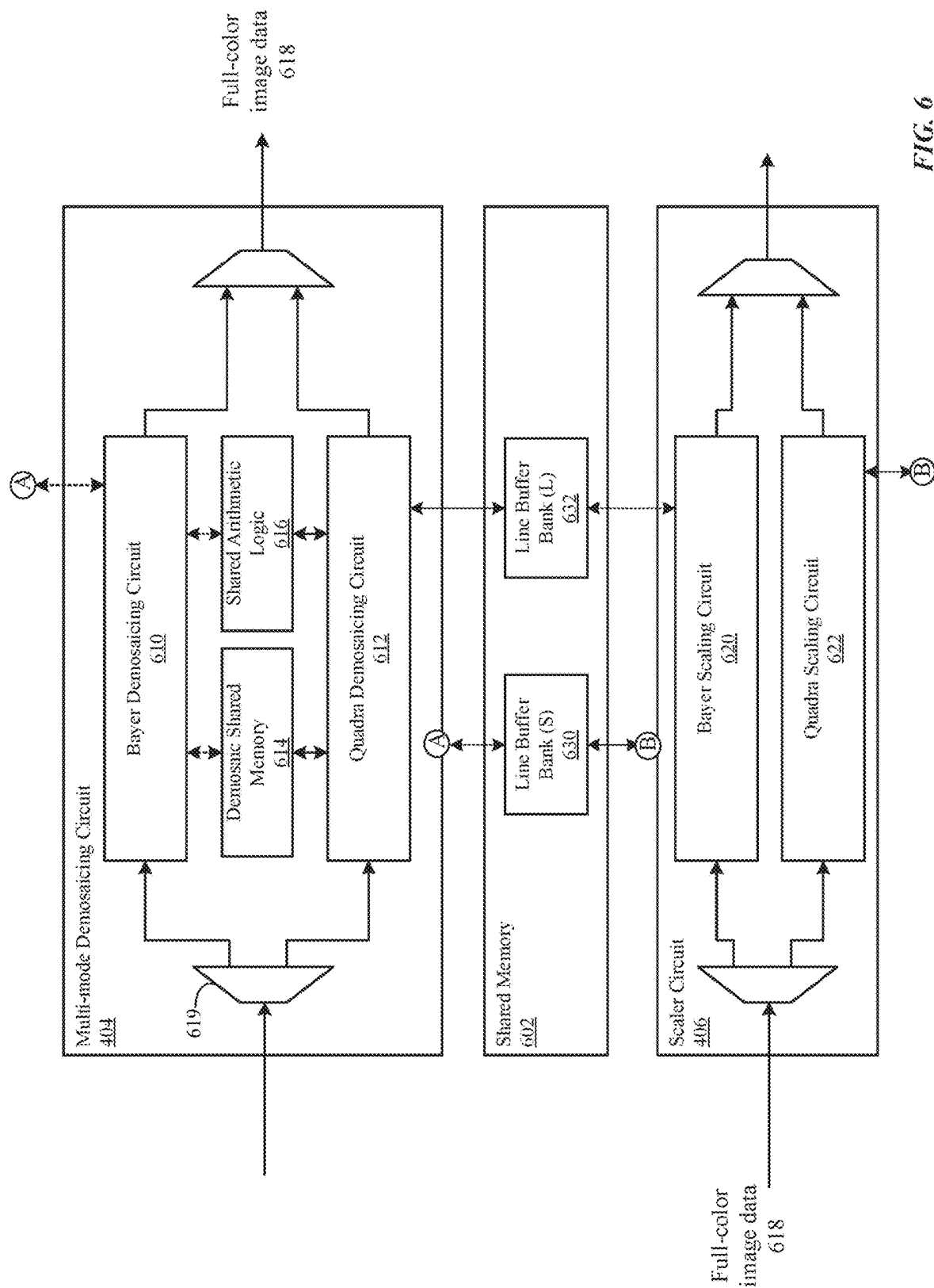


FIG. 6

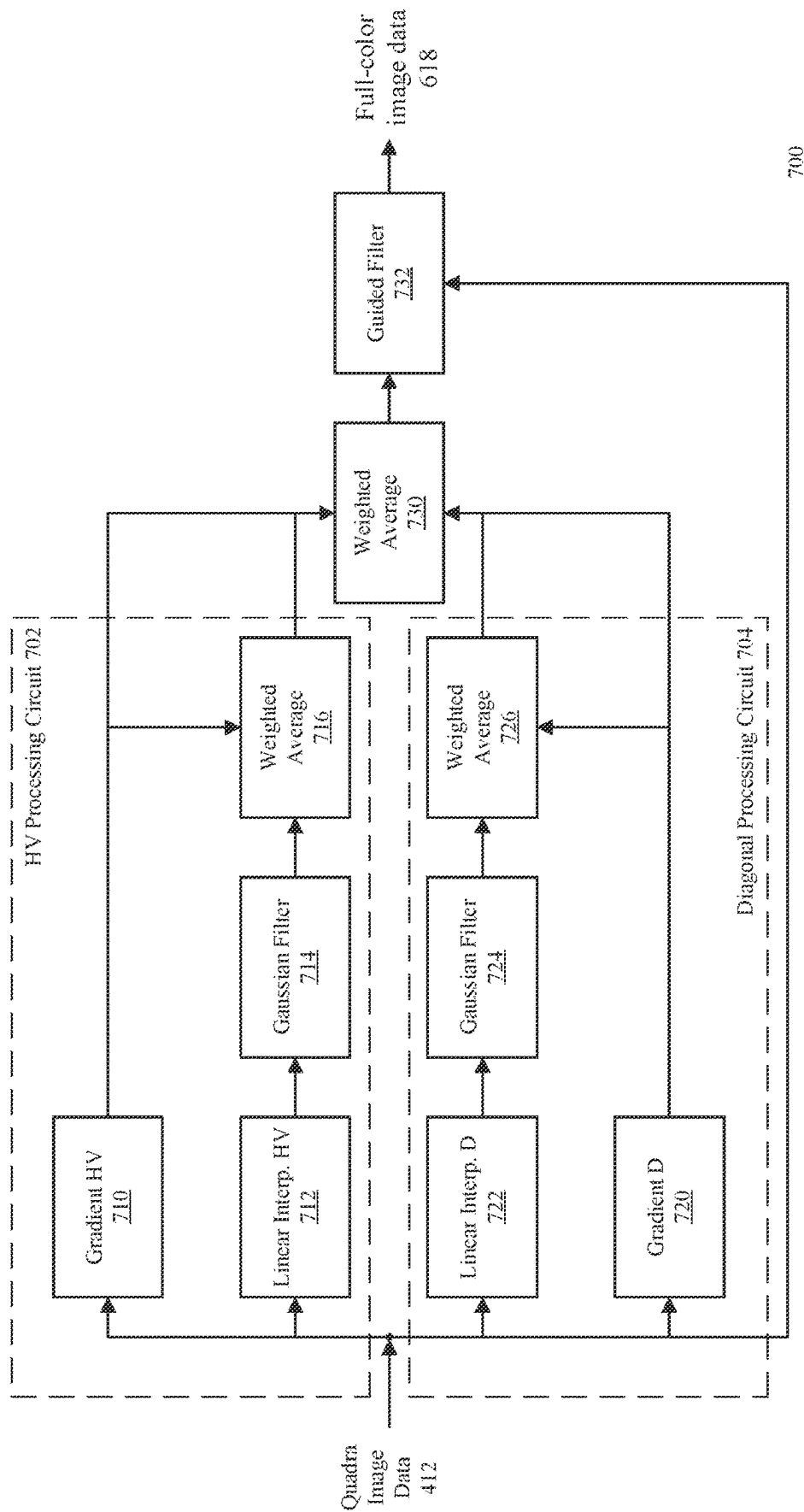


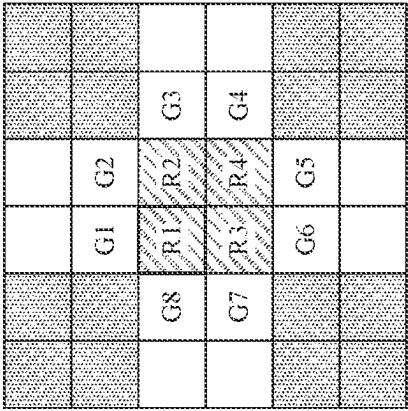
FIG. 7



802



804



806

FIG. 8

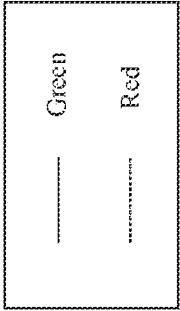
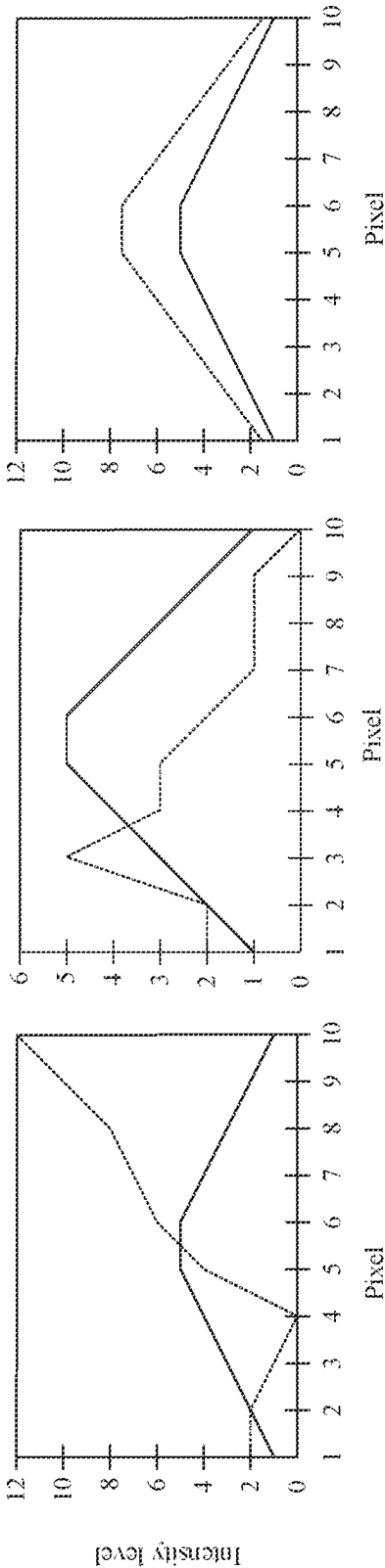


FIG. 9

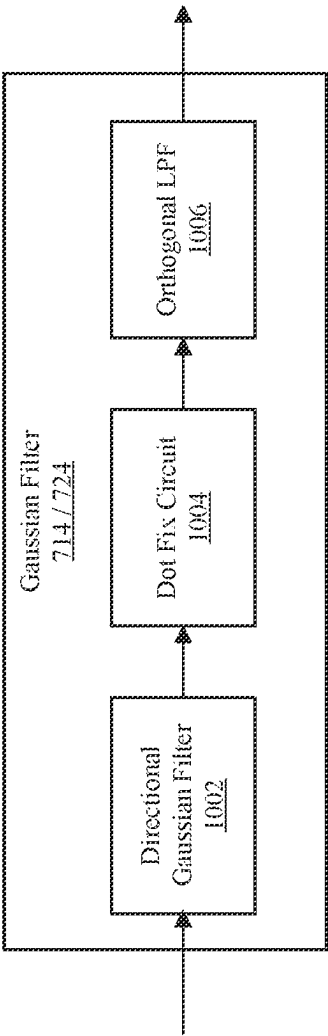


FIG. 10

		0.25
	0.5	
0.25		

Diagonal 2
1108

		0.25
0.25	0.5	

Diagonal 1
1106

	0.25	
	0.5	
	0.25	

Vertical
1104

		0.25
0.25	0.5	0.25

Horizontal
1102

FIG. 11

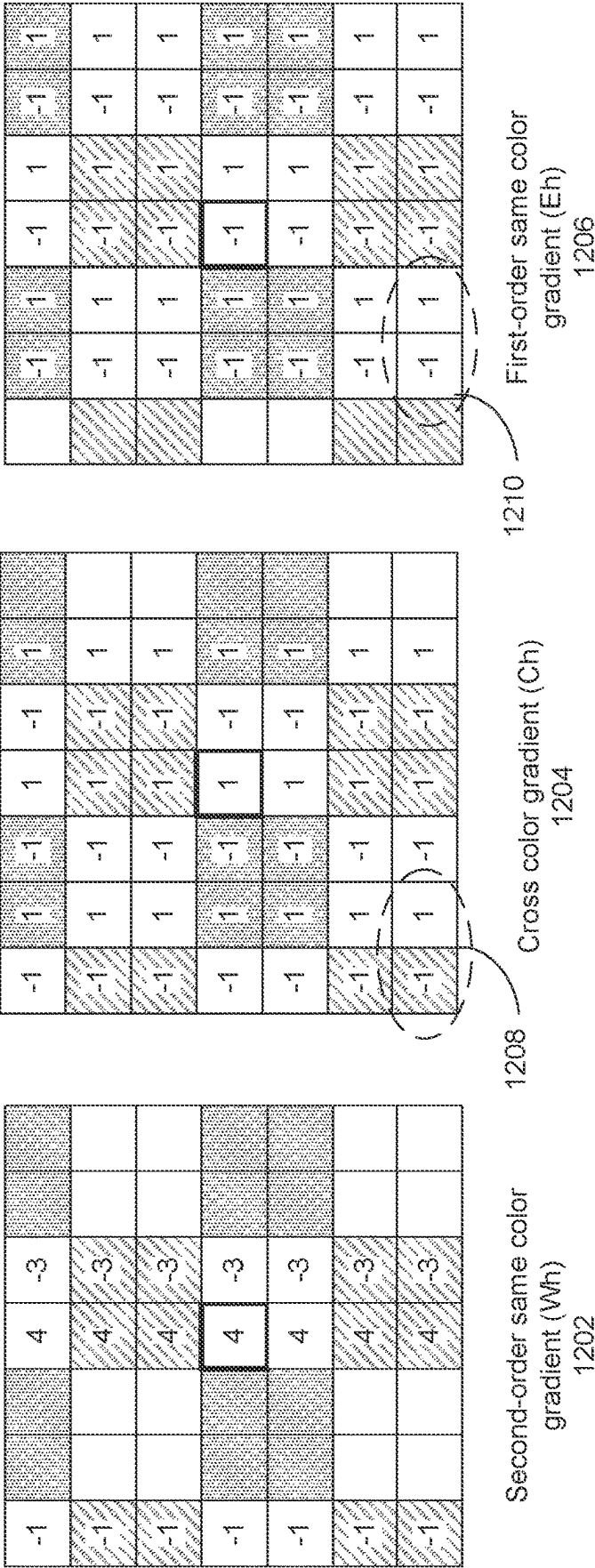


FIG. 12

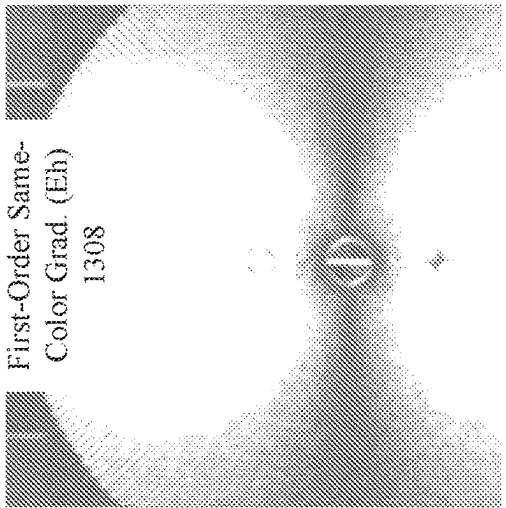
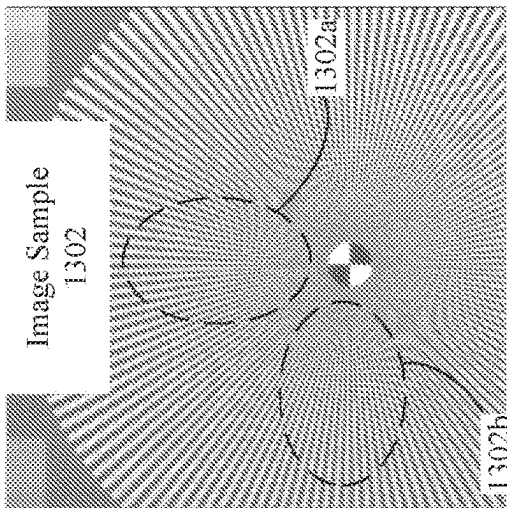
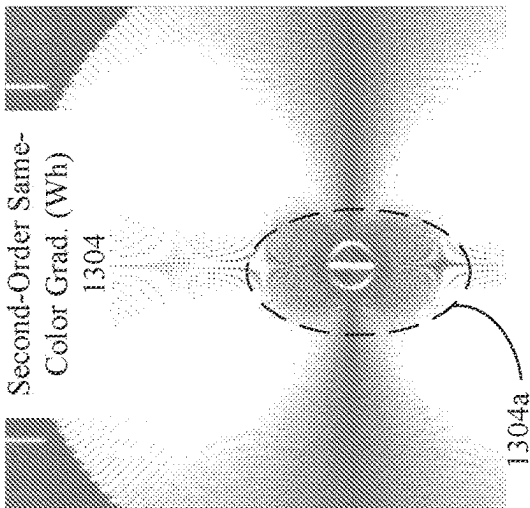
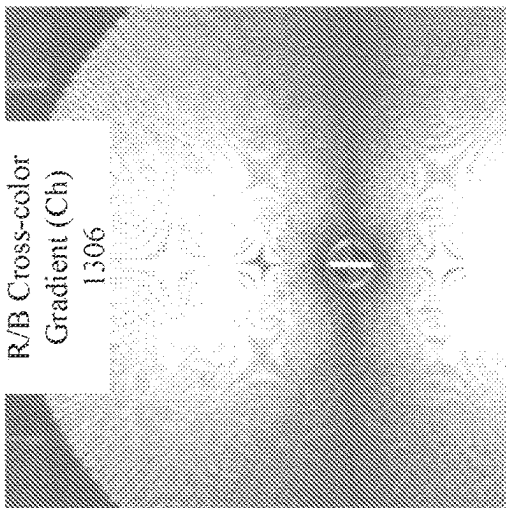


FIG. 13

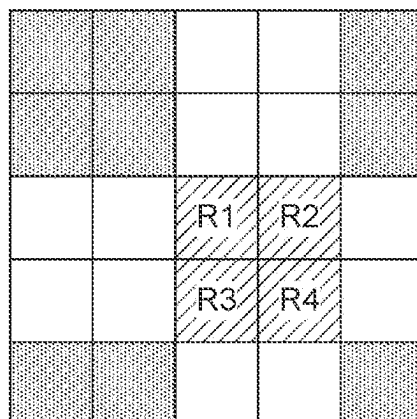
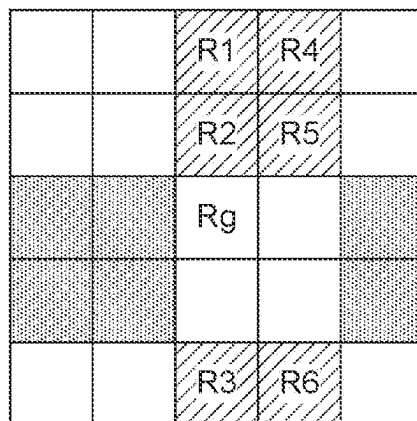
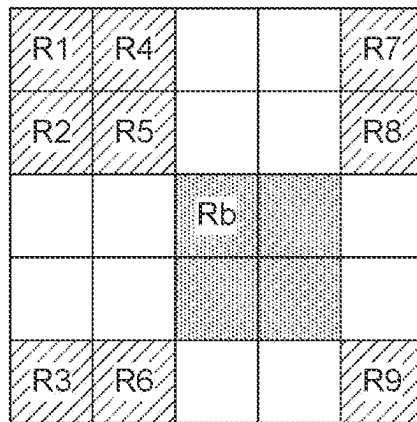
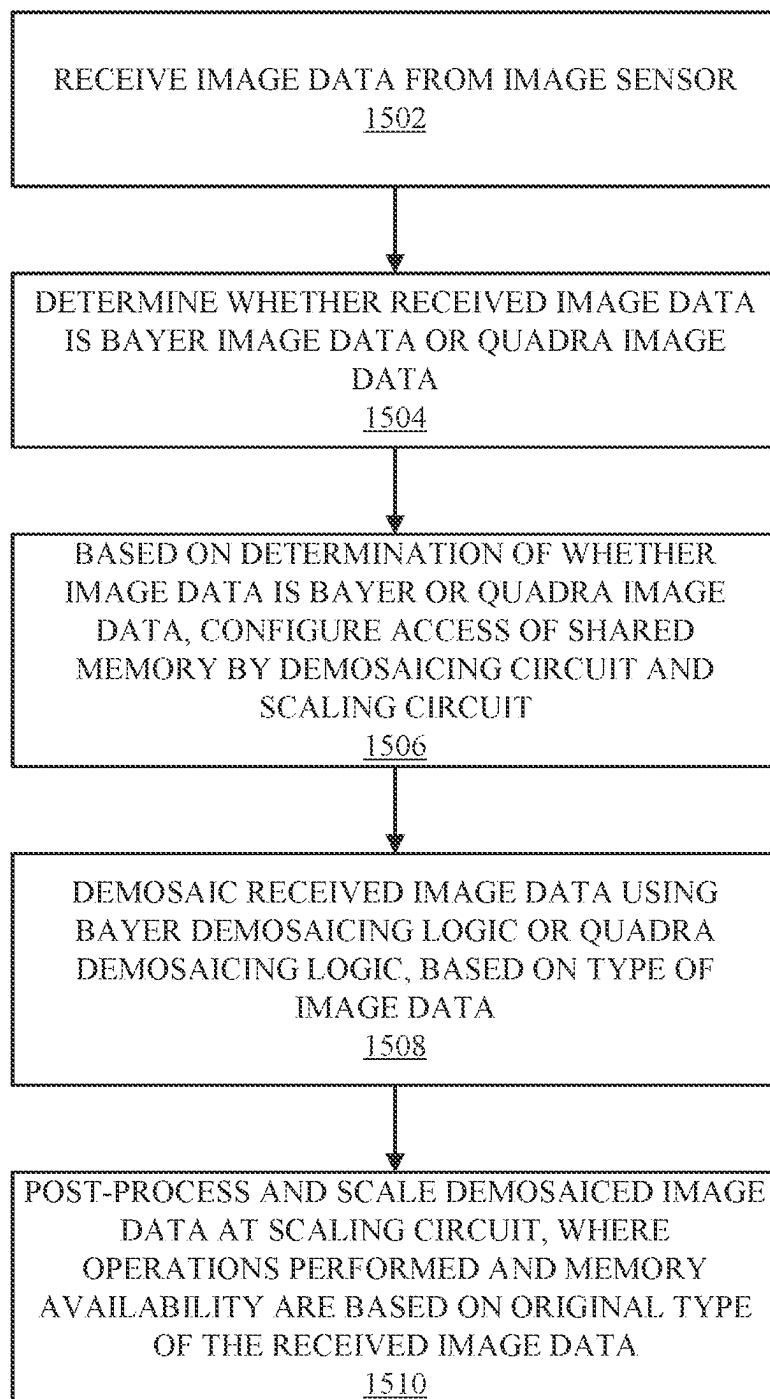


FIG. 14

*FIG. 15*

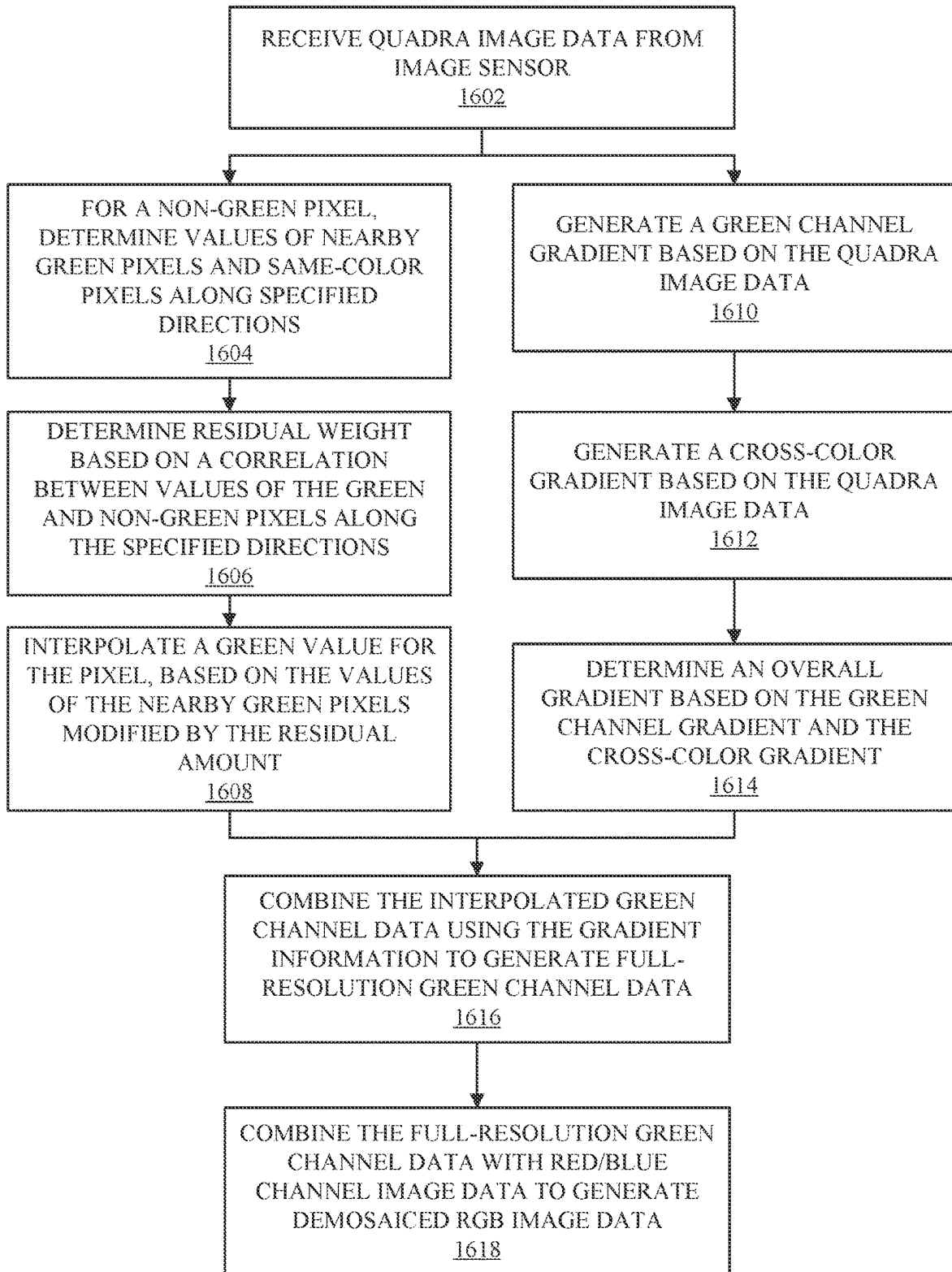


FIG. 16

1

DEMOSAICING CIRCUIT FOR DEMOSAICING QUAD BAYER RAW IMAGE DATA

CROSS-REFERENCE TO RELATED APPLICATIONS

The application is a continuation and claims benefit of U.S. patent application Ser. No. 17/578,127 filed on Jan. 18, 2022, the content of which is incorporated by reference in its entirety.

BACKGROUND

1. Field of the Disclosure

The present disclosure relates to a circuit for processing images and more specifically to image demosaicing.

2. Description of the Related Arts

Image data captured by an image sensor or received from other data sources is often processed in an image processing pipeline before further processing or consumption. For example, raw image data may be corrected, filtered, or otherwise modified before being provided to subsequent components such as a video encoder. To perform corrections or enhancements for captured image data, various components, unit stages or modules may be employed.

Such an image processing pipeline may be structured so that corrections or enhancements to the captured image data can be performed in an expedient way without consuming other system resources. Although many image processing algorithms may be performed by executing software programs on a central processing unit (CPU), execution of such programs on the CPU would consume significant bandwidth of the CPU and other peripheral resources as well as increase power consumption. Hence, image processing pipelines are often implemented as a hardware component separate from the CPU and dedicated to perform one or more image processing algorithms.

Image sensors typically capture image data using a color filter array, resulting in the raw image data where each pixel is associated with a particular color channel. Image processing pipelines may include circuitry for demosaicing raw image data to generate full-color image data, where different circuits may be used for different raw image formats.

SUMMARY

Embodiments relate to a circuit for processing image data. The circuit comprises a first demosaicing circuit configured to receive first image data in a first raw image format, and to demosaic the first image data to generate first demosaiced image data, as well as a second demosaicing circuit configured to receive second image data in a second raw image format, and to demosaic the second image data to generate second demosaiced image data. The second demosaicing circuit demosaics the second image data by interpolating a green channel of the second image data along one or more directions to yield an interpolated green channel, generating a gradient of the second image data, modifying the interpolated green channel based on the gradient to generate full-resolution green channel image data, and combining the full-resolution green channel image data with red and blue channel image data to generate the second demosaiced image data.

2

In one or more embodiments, the circuit further comprises a logic circuit configured to receive the first image data and the second image data from an image sensor, and to provide the first image data to the first demosaicing circuit, and the second image data to the second demosaicing circuit.

In some embodiments, the first raw image format is a Bayer raw image format, and the second raw image format is a Quad Bayer raw image format.

In some embodiments, the first demosaiced image data and the second demosaiced image data are both RGB image data.

In some embodiments, the first demosaicing circuit and the second demosaicing circuit utilized a shared set of line buffers for generating the first demosaiced image data from the first image data and the second demosaiced image from the second image data, respectively.

In some embodiments, the circuit further shares a second shared set of line buffers with a scaler circuit configured to perform post-processing on the first and second demosaiced image data.

BRIEF DESCRIPTION OF THE DRAWINGS

FIG. 1 is a high-level diagram of an electronic device, according to one embodiment.

FIG. 2 is a block diagram illustrating components in the electronic device, according to one embodiment.

FIG. 3 is a block diagram illustrating image processing pipelines implemented using an image signal processor, according to one embodiment.

FIG. 4 is a block diagram illustrating a demosaic processing circuit, according to one embodiment.

FIGS. 5A and 5B illustrate examples of raw image data in the Bayer image format and Quad Bayer image format, in accordance with some embodiments.

FIG. 6 is a block diagram of a more detailed view of the multi-mode demosaicing circuit and the scaling circuit, in accordance with some embodiments.

FIG. 7 illustrates a block diagram of a Quad Bayer demosaicing circuit, in accordance with some embodiments.

FIG. 8 illustrates examples of interpolation kernels that may be used to perform linear interpolation, in accordance with some embodiments.

FIG. 9 illustrates example diagrams of different amount of residual values added when interpolating image data, in accordance with some embodiments.

FIG. 10 illustrates a block diagram of a Gaussian filter used in the Quad Bayer demosaicing circuit, in accordance with some embodiments.

FIG. 11 illustrates examples of different kernels that may be used by orthogonal LPF, in accordance with some embodiments.

FIG. 12 illustrates examples of filter windows that may be used by the gradient calculation circuits, in accordance with some embodiments.

FIG. 13 illustrates an example of how the different filter windows for determining same-color gradient, cross-color gradient, and gradient error are combined to generate a final gradient value, in accordance with some embodiments.

FIG. 14 illustrates examples of guided filter windows that may be used to generate a red value for different types of pixels in a Quad Bayer image, in accordance with some embodiments.

FIG. 15 is a flowchart of a process for multi-mode demosaicing and scaling of image data, in accordance with some embodiments.

FIG. 16 is a flowchart of a process for demosaicing Quad Bayer image data, in accordance with some embodiments.

The figures depict, and the detailed description describes, various non-limiting embodiments for purposes of illustration only.

DETAILED DESCRIPTION

Reference will now be made in detail to embodiments, examples of which are illustrated in the accompanying drawings. In the following detailed description, numerous specific details are set forth in order to provide a thorough understanding of the various described embodiments. However, the described embodiments may be practiced without these specific details. In other instances, well-known methods, procedures, components, circuits, and networks have not been described in detail so as not to unnecessarily obscure aspects of the embodiments.

Embodiments relate to a multi-mode demosaicing circuit able to receive and demosaic image data in a different raw image formats (e.g., a first raw image format corresponding to Bayer image data, and a second raw image format corresponding to Quad Bayer image data). The multi-mode demosaicing circuit comprises different circuitry for demosaicing different image formats that access a shared working memory. In addition, the multi-mode demosaicing circuit shares memory with a post-processing and scaling circuit configured to perform subsequent post-processing and/or scaling of the demosaiced image data, in which the operations of the post-processing and scaling circuit are modified based on the original raw image format of the demosaiced image data to use different amounts of the shared memory, to compensate for additional memory utilized by the multi-mode demosaicing circuit when demosaicing certain types of image data (e.g., Quad Bayer image data). As such, the ability to demosaic raw image data of either the first or second raw image format is implemented without increasing an amount of memory on the chip relative to an amount of memory for performing demosaicing and subsequent processing for only image data of the first raw image format.

Exemplary Electronic Device

Embodiments of electronic devices, user interfaces for such devices, and associated processes for using such devices are described. In some embodiments, the device is a portable communications device, such as a mobile telephone, that also contains other functions, such as personal digital assistant (PDA) and/or music player functions. Exemplary embodiments of portable multifunction devices include, without limitation, the iPhone®, iPod Touch®, Apple Watch®, and iPad® devices from Apple Inc. of Cupertino, California. Other portable electronic devices, such as wearables, laptops or tablet computers, are optionally used. In some embodiments, the device is not a portable communications device, but is a desktop computer or other computing device that is not designed for portable use. In some embodiments, the disclosed electronic device may include a touch sensitive surface (e.g., a touch screen display and/or a touch pad). An example electronic device described below in conjunction with FIG. 1 (e.g., device 100) may include a touch-sensitive surface for receiving user input. The electronic device may also include one or more other physical user-interface devices, such as a physical keyboard, a mouse and/or a joystick.

Figure (FIG. 1 is a high-level diagram of an electronic device 100, according to one embodiment. Device 100 may include one or more physical buttons, such as a “home” or menu button 104. Menu button 104 is, for example, used to

navigate to any application in a set of applications that are executed on device 100. In some embodiments, menu button 104 includes a fingerprint sensor that identifies a fingerprint on menu button 104. The fingerprint sensor may be used to determine whether a finger on menu button 104 has a fingerprint that matches a fingerprint stored for unlocking device 100. Alternatively, in some embodiments, menu button 104 is implemented as a soft key in a graphical user interface (GUI) displayed on a touch screen.

In some embodiments, device 100 includes touch screen 150, menu button 104, push button 106 for powering the device on/off and locking the device, volume adjustment buttons 108, Subscriber Identity Module (SIM) card slot 110, head set jack 112, and docking/charging external port 124. Push button 106 may be used to turn the power on/off on the device by depressing the button and holding the button in the depressed state for a predefined time interval; to lock the device by depressing the button and releasing the button before the predefined time interval has elapsed; and/or to unlock the device or initiate an unlock process. In an alternative embodiment, device 100 also accepts verbal input for activation or deactivation of some functions through microphone 113. The device 100 includes various components including, but not limited to, a memory (which may include one or more computer readable storage mediums), a memory controller, one or more central processing units (CPUs), a peripherals interface, an RF circuitry, an audio circuitry, speaker 111, microphone 113, input/output (I/O) subsystem, and other input or control devices. Device 100 may include one or more image sensors 164, one or more proximity sensors 166, and one or more accelerometers 168. Device 100 may include more than one type of image sensors 164. Each type may include more than one image sensor 164. For example, one type of image sensors 164 may be cameras and another type of image sensors 164 may be infrared sensors that may be used for face recognition. In addition or alternatively, the image sensors 164 may be associated with different lens configuration. For example, device 100 may include rear image sensors, one with a wide-angle lens and another with a telephoto lens. The device 100 may include components not shown in FIG. 1 such as an ambient light sensor, a dot projector and a flood illuminator.

Device 100 is only one example of an electronic device, and device 100 may have more or fewer components than listed above, some of which may be combined into a component or have a different configuration or arrangement. The various components of device 100 listed above are embodied in hardware, software, firmware or a combination thereof, including one or more signal processing and/or application specific integrated circuits (ASICs). While the components in FIG. 1 are shown as generally located on the same side as the touch screen 150, one or more components may also be located on an opposite side of device 100. For example, the front side of device 100 may include an infrared image sensor 164 for face recognition and another image sensor 164 as the front camera of device 100. The back side of device 100 may also include additional two image sensors 164 as the rear cameras of device 100.

FIG. 2 is a block diagram illustrating components in device 100, according to one embodiment. Device 100 may perform various operations including image processing. For this and other purposes, the device 100 may include, among other components, image sensor 202, system-on-a chip (SOC) component 204, system memory 230, persistent storage (e.g., flash memory) 228, orientation sensor 234, and display 216. The components as illustrated in FIG. 2 are

merely illustrative. For example, device **100** may include other components (such as speaker or microphone) that are not illustrated in FIG. 2. Further, some components (such as orientation sensor **234**) may be omitted from device **100**.

Image sensors **202** are components for capturing image data. Each of the image sensors **202** may be embodied, for example, as a complementary metal-oxide-semiconductor (CMOS) active-pixel sensor, a camera, video camera, or other devices. Image sensors **202** generate raw image data that is sent to SOC component **204** for further processing. In some embodiments, the image data processed by SOC component **204** is displayed on display **216**, stored in system memory **230**, persistent storage **228** or sent to a remote computing device via network connection. The raw image data generated by image sensors **202** may be in a Bayer color filter array (CFA) pattern (hereinafter also referred to as “Bayer pattern”). An image sensor **202** may also include optical and mechanical components that assist image sensing components (e.g., pixels) to capture images. The optical and mechanical components may include an aperture, a lens system, and an actuator that controls the lens position of the image sensor **202**.

Motion sensor **234** is a component or a set of components for sensing motion of device **100**. Motion sensor **234** may generate sensor signals indicative of orientation and/or acceleration of device **100**. The sensor signals are sent to SOC component **204** for various operations such as turning on device **100** or rotating images displayed on display **216**.

Display **216** is a component for displaying images as generated by SOC component **204**. Display **216** may include, for example, a liquid crystal display (LCD) device or an organic light emitting diode (OLED) device. Based on data received from SOC component **204**, display **216** may display various images, such as menus, selected operating parameters, images captured by image sensor **202** and processed by SOC component **204**, and/or other information received from a user interface of device **100** (not shown).

System memory **230** is a component for storing instructions for execution by SOC component **204** and for storing data processed by SOC component **204**. System memory **230** may be embodied as any type of memory including, for example, dynamic random access memory (DRAM), synchronous DRAM (SDRAM), double data rate (DDR, DDR2, DDR3, etc.) RAMBUS DRAM (RDRAM), static RAM (SRAM) or a combination thereof. In some embodiments, system memory **230** may store pixel data or other image data or statistics in various formats.

Persistent storage **228** is a component for storing data in a non-volatile manner. Persistent storage **228** retains data even when power is not available. Persistent storage **228** may be embodied as read-only memory (ROM), flash memory or other non-volatile random access memory devices.

SOC component **204** is embodied as one or more integrated circuit (IC) chip and performs various data processing processes. SOC component **204** may include, among other subcomponents, image signal processor (ISP) **206**, a central processor unit (CPU) **208**, a network interface **210**, motion sensor interface **212**, display controller **214**, graphics processor (GPU) **220**, memory controller **222**, video encoder **224**, storage controller **226**, and various other input/output (I/O) interfaces **218**, and bus **232** connecting these subcomponents. SOC component **204** may include more or fewer subcomponents than those shown in FIG. 2.

ISP **206** is hardware that performs various stages of an image processing pipeline. In some embodiments, ISP **206** may receive raw image data from image sensor **202**, and

process the raw image data into a form that is usable by other subcomponents of SOC component **204** or components of device **100**. ISP **206** may perform various image-manipulation operations such as image translation operations, horizontal and vertical scaling, color space conversion and/or image stabilization transformations, as described below in detail with reference to FIG. 3.

CPU **208** may be embodied using any suitable instruction set architecture, and may be configured to execute instructions defined in that instruction set architecture. CPU **208** may be general-purpose or embedded processors using any of a variety of instruction set architectures (ISAs), such as the x86, PowerPC, SPARC, RISC, ARM or MIPS ISAs, or any other suitable ISA. Although a single CPU is illustrated in FIG. 2, SOC component **204** may include multiple CPUs. In multiprocessor systems, each of the CPUs may commonly, but not necessarily, implement the same ISA.

Graphics processing unit (GPU) **220** is graphics processing circuitry for performing operations on graphical data. For example, GPU **220** may render objects to be displayed into a frame buffer (e.g., one that includes pixel data for an entire frame). GPU **220** may include one or more graphics processors that may execute graphics software to perform a part or all of the graphics operation, or hardware acceleration of certain graphics operations.

I/O interfaces **218** are hardware, software, firmware or combinations thereof for interfacing with various input/output components in device **100**. I/O components may include devices such as keypads, buttons, audio devices, and sensors such as a global positioning system. I/O interfaces **218** process data for sending data to such I/O components or process data received from such I/O components.

Network interface **210** is a subcomponent that enables data to be exchanged between devices **100** and other devices via one or more networks (e.g., carrier or agent devices). For example, video or other image data may be received from other devices via network interface **210** and be stored in system memory **230** for subsequent processing (e.g., via a back-end interface to image signal processor **206**, such as discussed below in FIG. 3) and display. The networks may include, but are not limited to, Local Area Networks (LANs) (e.g., an Ethernet or corporate network) and Wide Area Networks (WANs). The image data received via network interface **210** may undergo image processing processes by ISP **206**.

Motion sensor interface **212** is circuitry for interfacing with motion sensor **234**. Motion sensor interface **212** receives sensor information from motion sensor **234** and processes the sensor information to determine the orientation or movement of the device **100**.

Display controller **214** is circuitry for sending image data to be displayed on display **216**. Display controller **214** receives the image data from ISP **206**, CPU **208**, graphic processor or system memory **230** and processes the image data into a format suitable for display on display **216**.

Memory controller **222** is circuitry for communicating with system memory **230**. Memory controller **222** may read data from system memory **230** for processing by ISP **206**, CPU **208**, GPU **220** or other subcomponents of SOC component **204**. Memory controller **222** may also write data to system memory **230** received from various subcomponents of SOC component **204**.

Video encoder **224** is hardware, software, firmware or a combination thereof for encoding video data into a format suitable for storing in persistent storage **228** or for passing the data to network interface **210** for transmission over a network to another device.

In some embodiments, one or more subcomponents of SOC component 204 or some functionality of these sub-components may be performed by software components executed on ISP 206, CPU 208 or GPU 220. Such software components may be stored in system memory 230, persistent storage 228 or another device communicating with device 100 via network interface 210.

Image data or video data may flow through various data paths within SOC component 204. In one example, raw image data may be generated from the image sensors 202 and processed by ISP 206, and then sent to system memory 230 via bus 232 and memory controller 222. After the image data is stored in system memory 230, it may be accessed by video encoder 224 for encoding or by display 116 for displaying via bus 232.

In another example, image data is received from sources other than the image sensors 202. For example, video data may be streamed, downloaded, or otherwise communicated to the SOC component 204 via wired or wireless network. The image data may be received via network interface 210 and written to system memory 230 via memory controller 222. The image data may then be obtained by ISP 206 from system memory 230 and processed through one or more image processing pipeline stages, as described below in detail with reference to FIG. 3. The image data may then be returned to system memory 230 or be sent to video encoder 224, display controller 214 (for display on display 216), or storage controller 226 for storage at persistent storage 228.

Example Image Signal Processing Pipelines

FIG. 3 is a block diagram illustrating image processing pipelines implemented using ISP 206, according to one embodiment. In the embodiment of FIG. 3, ISP 206 is coupled to an image sensor system 201 that includes one or more image sensors 202A through 202N (hereinafter collectively referred to as “image sensors 202” or also referred individually as “image sensor 202”) to receive raw image data. The image sensor system 201 may include one or more sub-systems that control the image sensors 202 individually. In some cases, each image sensor 202 may operate independently while, in other cases, the image sensors 202 may share some components. For example, in one embodiment, two or more image sensors 202 may share the same circuit board that controls the mechanical components of the image sensors (e.g., actuators that change the lens positions of each image sensor). The image sensing components of an image sensor 202 may include different types of image sensing components that may provide raw image data in different forms to the ISP 206. For example, in one embodiment, the image sensing components may include a plurality of focus pixels that are used for auto-focusing and a plurality of image pixels that are used for capturing images. In another embodiment, the image sensing pixels may be used for both auto-focusing and image capturing purposes.

ISP 206 implements an image processing pipeline which may include a set of stages that process image information from creation, capture or receipt to output. ISP 206 may include, among other components, sensor interface 302, central control 320, front-end pipeline stages 330, back-end pipeline stages 340, image statistics module 304, vision module 322, back-end interface 342, output interface 316, and auto-focus circuits 350A through 350N (hereinafter collectively referred to as “auto-focus circuits 350” or referred individually as “auto-focus circuits 350”). ISP 206 may include other components not illustrated in FIG. 3 or may omit one or more components illustrated in FIG. 3.

In one or more embodiments, different components of ISP 206 process image data at different rates. In the embodiment

of FIG. 3, front-end pipeline stages 330 (e.g., raw processing stage 306 and resample processing stage 308) may process image data at an initial rate. Thus, the various different techniques, adjustments, modifications, or other processing operations performed by these front-end pipeline stages 330 at the initial rate. For example, if the front-end pipeline stages 330 process 2 pixels per clock cycle, then raw processing stage 306 operations (e.g., black level compensation, highlight recovery and defective pixel correction) may process 2 pixels of image data at a time. In contrast, one or more back-end pipeline stages 340 may process image data at a different rate less than the initial data rate. For example, in the embodiment of FIG. 3, back-end pipeline stages 340 (e.g., noise processing stage 310, color processing stage 312, and output rescale 314) may be processed at a reduced rate (e.g., 1 pixel per clock cycle). Raw image data captured by image sensors 202 may be transmitted to different components of ISP 206 in different manners. In one embodiment, raw image data corresponding to the focus pixels may be sent to the auto-focus circuits 350 while raw image data corresponding to the image pixels may be sent to the sensor interface 302. In another embodiment, raw image data corresponding to both types of pixels may simultaneously be sent to both the auto-focus circuits 350 and the sensor interface 302.

Auto-focus circuits 350 may include a hardware circuit that analyzes raw image data to determine an appropriate lens position of each image sensor 202. In one embodiment, the raw image data may include data that is transmitted from image sensing pixels that specialize in image focusing. In another embodiment, raw image data from image capture pixels may also be used for auto-focusing purpose. An auto-focus circuit 350 may perform various image processing operations to generate data that determines the appropriate lens position. The image processing operations may include cropping, binning, image compensation, scaling to generate data that is used for auto-focusing purpose. The auto-focusing data generated by auto-focus circuits 350 may be fed back to the image sensor system 201 to control the lens positions of the image sensors 202. For example, an image sensor 202 may include a control circuit that analyzes the auto-focusing data to determine a command signal that is sent to an actuator associated with the lens system of the image sensor to change the lens position of the image sensor. The data generated by the auto-focus circuits 350 may also be sent to other components of the ISP 206 for other image processing purposes. For example, some of the data may be sent to image statistics 304 to determine information regarding auto-exposure.

The auto-focus circuits 350 may be individual circuits that are separate from other components such as image statistics 304, sensor interface 302, front-end 330 and back-end 340. This allows the ISP 206 to perform auto-focusing analysis independent of other image processing pipelines. For example, the ISP 206 may analyze raw image data from the image sensor 202A to adjust the lens position of image sensor 202A using the auto-focus circuit 350A while performing downstream image processing of the image data from image sensor 202B simultaneously. In one embodiment, the number of auto-focus circuits 350 may correspond to the number of image sensors 202. In other words, each image sensor 202 may have a corresponding auto-focus circuit that is dedicated to the auto-focusing of the image sensor 202. The device 100 may perform auto focusing for different image sensors 202 even if one or more image sensors 202 are not in active use. This allows a seamless transition between two image sensors 202 when the device

100 switches from one image sensor 202 to another. For example, in one embodiment, a device 100 may include a wide-angle camera and a telephoto camera as a dual back camera system for photo and image processing. The device 100 may display images captured by one of the dual cameras and may switch between the two cameras from time to time. The displayed images may seamlessly transition from image data captured by one image sensor 202 to image data captured by another image sensor 202 without waiting for the second image sensor 202 to adjust its lens position because two or more auto-focus circuits 350 may continuously provide auto-focus data to the image sensor system 201.

Raw image data captured by different image sensors 202 may also be transmitted to a sensor interface 302. Sensor interface 302 receives raw image data from image sensor 202 and processes the raw image data into an image data processable by other stages in the pipeline. Sensor interface 302 may perform various preprocessing operations, such as image cropping, binning or scaling to reduce image data size. In some embodiments, pixels are sent from the image sensor 202 to sensor interface 302 in raster order (i.e., horizontally, line by line). The subsequent processes in the pipeline may also be performed in raster order and the result may also be output in raster order. Although only a single image sensor and a single sensor interface 302 are illustrated in FIG. 3, when more than one image sensor is provided in device 100, a corresponding number of sensor interfaces may be provided in ISP 206 to process raw image data from each image sensor.

Front-end pipeline stages 330 process image data in raw or full-color domains. Front-end pipeline stages 330 may include, but are not limited to, raw processing stage 306 and resample processing stage 308. A raw image data may be in Bayer raw format, for example. In Bayer raw image format, pixel data with values specific to a particular color (instead of all colors) is provided in each pixel. In an image capturing sensor, image data is typically provided in a Bayer pattern. Raw processing stage 306 may process image data in a Bayer raw format.

The operations performed by raw processing stage 306 include, but are not limited, sensor linearization, black level compensation, fixed pattern noise reduction, defective pixel correction, raw noise filtering, lens shading correction, white balance gain, and highlight recovery. Sensor linearization refers to mapping non-linear image data to linear space for other processing. Black level compensation refers to providing digital gain, offset and clip independently for each color component (e.g., Gr, R, B, Gb) of the image data. Fixed pattern noise reduction refers to removing offset fixed pattern noise and gain fixed pattern noise by subtracting a dark frame from an input image and multiplying different gains to pixels. Defective pixel correction refers to detecting defective pixels, and then replacing defective pixel values. Raw noise filtering refers to reducing noise of image data by averaging neighbor pixels that are similar in brightness. Highlight recovery refers to estimating pixel values for those pixels that are clipped (or nearly clipped) from other channels. Lens shading correction refers to applying a gain per pixel to compensate for a drop-off in intensity roughly proportional to a distance from a lens optical center. White balance gain refers to providing digital gains for white balance, offset and clip independently for all color components (e.g., Gr, R, B, Gb in Bayer format). Components of ISP 206 may convert raw image data into image data in

full-color domain, and thus, raw processing stage 306 may process image data in the full-color domain in addition to or instead of raw image data.

Resample processing stage 308 performs various operations to convert, resample, or scale image data received from raw processing stage 306. Operations performed by resample processing stage 308 may include, but not limited to, demosaic operation, per-pixel color correction operation, Gamma mapping operation, color space conversion and downscaling or sub-band splitting. Demosaic operation refers to converting or interpolating missing color samples from raw image data (for example, in a Bayer pattern) to output image data into a full-color domain. Demosaic operation may include low pass directional filtering on the interpolated samples to obtain full-color pixels. Per-pixel color correction operation refers to a process of performing color correction on a per-pixel basis using information about relative noise standard deviations of each color channel to correct color without amplifying noise in the image data. Gamma mapping refers to converting image data from input image data values to output data values to perform gamma correction. For the purpose of Gamma mapping, lookup tables (or other structures that index pixel values to another value) for different color components or channels of each pixel (e.g., a separate lookup table for R, G, and B color components) may be used. Color space conversion refers to converting color space of an input image data into a different format. In one embodiment, resample processing stage 308 converts RGB format into YCbCr format for further processing.

Central control module 320 may control and coordinate overall operation of other components in ISP 206. Central control module 320 performs operations including, but not limited to, monitoring various operating parameters (e.g., logging clock cycles, memory latency, quality of service, and state information), updating or managing control parameters for other components of ISP 206, and interfacing with sensor interface 302 to control the starting and stopping of other components of ISP 206. For example, central control module 320 may update programmable parameters for other components in ISP 206 while the other components are in an idle state. After updating the programmable parameters, central control module 320 may place these components of ISP 206 into a run state to perform one or more operations or tasks. Central control module 320 may also instruct other components of ISP 206 to store image data (e.g., by writing to system memory 230 in FIG. 2) before, during, or after resample processing stage 308. In this way full-resolution image data in raw or full-color domain format may be stored in addition to or instead of processing the image data output from resample processing stage 308 through backend pipeline stages 340.

Image statistics module 304 performs various operations to collect statistic information associated with the image data. The operations for collecting statistics information may include, but not limited to, sensor linearization, replace patterned defective pixels, sub-sample raw image data, detect and replace non-patterned defective pixels, black level compensation, lens shading correction, and inverse black level compensation. After performing one or more of such operations, statistics information such as 3A statistics (Auto white balance (AWB), auto exposure (AE), histograms (e.g., 2D color or component) and any other image data information may be collected or tracked. In some embodiments, certain pixels' values, or areas of pixel values may be excluded from collections of certain statistical data when preceding operations identify clipped pixels. Although

only a single statistics module **304** is illustrated in FIG. 3, multiple image statistics modules may be included in ISP **206**. For example, each image sensor **202** may correspond to an individual image statistics unit **304**. In such embodiments, each statistic module may be programmed by central control module **320** to collect different information for the same or different image data.

Vision module **322** performs various operations to facilitate computer vision operations at CPU **208** such as facial detection in image data. The vision module **322** may perform various operations including pre-processing, global tone-mapping and Gamma correction, vision noise filtering, resizing, keypoint detection, generation of histogram-of-orientation gradients (HOG) and normalized cross correlation (NCC). The pre-processing may include subsampling or binning operation and computation of luminance if the input image data is not in YCbCr format. Global mapping and Gamma correction can be performed on the pre-processed data on luminance image. Vision noise filtering is performed to remove pixel defects and reduce noise present in the image data, and thereby, improve the quality and performance of subsequent computer vision algorithms. Such vision noise filtering may include detecting and fixing dots or defective pixels, and performing bilateral filtering to reduce noise by averaging neighbor pixels of similar brightness. Various vision algorithms use images of different sizes and scales. Resizing of an image is performed, for example, by binning or linear interpolation operation. Keypoints are locations within an image that are surrounded by image patches well suited to matching in other images of the same scene or object. Such keypoints are useful in image alignment, computing camera pose and object tracking. Keypoint detection refers to the process of identifying such keypoints in an image. HOG provides descriptions of image patches for tasks in image analysis and computer vision. HOG can be generated, for example, by (i) computing horizontal and vertical gradients using a simple difference filter, (ii) computing gradient orientations and magnitudes from the horizontal and vertical gradients, and (iii) binning the gradient orientations. NCC is the process of computing spatial cross-correlation between a patch of an image and a kernel.

Back-end interface **342** receives image data from other image sources than image sensor **102** and forwards it to other components of ISP **206** for processing. For example, image data may be received over a network connection and be stored in system memory **230**. Back-end interface **342** retrieves the image data stored in system memory **230** and provides it to back-end pipeline stages **340** for processing. One of many operations that are performed by back-end interface **342** is converting the retrieved image data to a format that can be utilized by back-end processing stages **340**. For instance, back-end interface **342** may convert RGB, YCbCr 4:2:0, or YCbCr 4:2:2 formatted image data into YCbCr 4:4:4 color format.

Back-end pipeline stages **340** processes image data according to a particular full-color format (e.g., YCbCr 4:4:4 or RGB). In some embodiments, components of the back-end pipeline stages **340** may convert image data to a particular full-color format before further processing. Back-end pipeline stages **340** may include, among other stages, noise processing stage **310** and color processing stage **312**. Back-end pipeline stages **340** may include other stages not illustrated in FIG. 3.

Noise processing stage **310** performs various operations to reduce noise in the image data. The operations performed by noise processing stage **310** include, but are not limited to, color space conversion, gamma/de-gamma mapping, tem-

poral filtering, noise filtering, luma sharpening, and chroma noise reduction. The color space conversion may convert an image data from one color space format to another color space format (e.g., RGB format converted to YCbCr format). Gamma/de-gamma operation converts image data from input image data values to output data values to perform gamma correction or reverse gamma correction. Temporal filtering filters noise using a previously filtered image frame to reduce noise. For example, pixel values of a prior image frame are combined with pixel values of a current image frame. Noise filtering may include, for example, spatial noise filtering. Luma sharpening may sharpen luma values of pixel data while chroma suppression may attenuate chroma to gray (i.e. no color). In some embodiment, the luma sharpening and chroma suppression may be performed simultaneously with spatial noise filtering. The aggressiveness of noise filtering may be determined differently for different regions of an image. Spatial noise filtering may be included as part of a temporal loop implementing temporal filtering. For example, a previous image frame may be processed by a temporal filter and a spatial noise filter before being stored as a reference frame for a next image frame to be processed. In other embodiments, spatial noise filtering may not be included as part of the temporal loop for temporal filtering (e.g., the spatial noise filter may be applied to an image frame after it is stored as a reference image frame and thus the reference frame is not spatially filtered).

Color processing stage **312** may perform various operations associated with adjusting color information in the image data. The operations performed in color processing stage **312** include, but are not limited to, local tone mapping, gain/offset/clip, color correction, three-dimensional color lookup, gamma conversion, and color space conversion. Local tone mapping refers to spatially varying local tone curves in order to provide more control when rendering an image. For instance, a two-dimensional grid of tone curves (which may be programmed by the central control module **320**) may be bi-linearly interpolated such that smoothly varying tone curves are created across an image. In some embodiments, local tone mapping may also apply spatially varying and intensity varying color correction matrices, which may, for example, be used to make skies bluer while turning down blue in the shadows in an image. Digital gain/offset/clip may be provided for each color channel or component of image data. Color correction may apply a color correction transform matrix to image data. 3D color lookup may utilize a three dimensional array of color component output values (e.g., R, G, B) to perform advanced tone mapping, color space conversions, and other color transforms. Gamma conversion may be performed, for example, by mapping input image data values to output data values in order to perform gamma correction, tone mapping, or histogram matching. Color space conversion may be implemented to convert image data from one color space to another (e.g., RGB to YCbCr). Other processing techniques may also be performed as part of color processing stage **312** to perform other special image effects, including black and white conversion, sepia tone conversion, negative conversion, or solarize conversion.

Output rescale module **314** may resample, transform and correct distortion on the fly as the ISP **206** processes image data. Output rescale module **314** may compute a fractional input coordinate for each pixel and uses this fractional input coordinate to interpolate an output pixel via a polyphase resampling filter. A fractional input coordinate may be produced from a variety of possible transforms of an output

coordinate, such as resizing or cropping an image (e.g., via a simple horizontal and vertical scaling transform), rotating and shearing an image (e.g., via non-separable matrix transforms), perspective warping (e.g., via an additional depth transform) and per-pixel perspective divides applied piecewise in strips to account for changes in image sensor during image data capture (e.g., due to a rolling shutter), and geometric distortion correction (e.g., via computing a radial distance from the optical center in order to index an interpolated radial gain table, and applying a radial perturbation to a coordinate to account for a radial lens distortion).

Output rescale module 314 may apply transforms to image data as it is processed at output rescale module 314. Output rescale module 314 may include horizontal and vertical scaling components. The vertical portion of the design may implement a series of image data line buffers to hold the “support” needed by the vertical filter. As ISP 206 may be a streaming device, it may be that only the lines of image data in a finite-length sliding window of lines are available for the filter to use. Once a line has been discarded to make room for a new incoming line, the line may be unavailable. Output rescale module 314 may statistically monitor computed input Y coordinates over previous lines and use it to compute an optimal set of lines to hold in the vertical support window. For each subsequent line, output rescale module may automatically generate a guess as to the center of the vertical support window. In some embodiments, output rescale module 314 may implement a table of piecewise perspective transforms encoded as digital difference analyzer (DDA) steppers to perform a per-pixel perspective transformation between input image data and output image data in order to correct artifacts and motion caused by sensor motion during the capture of the image frame. Output rescale may provide image data via the output interface 316 to various other components of device 100, as discussed above with regard to FIGS. 1 and 2. In various embodiments, the functionality of components 302 through 350 may be performed in a different order than the order implied by the order of these functional units in the image processing pipeline illustrated in FIG. 3, or may be performed by different functional components than those illustrated in FIG. 3. Moreover, the various components as described in FIG. 3 may be embodied in various combinations of hardware, firmware or software.

Multi-Mode Demosaicing

FIG. 4 is a block diagram illustrating a demosaic processing circuit, according to one embodiment. In some embodiments, the demosaic processing circuit 402 is implemented as part of the resample processing stage 308 illustrated in FIG. 3.

The demosaic processing circuit 402 includes a multi-mode demosaicing circuit 404, a front-end scaler circuit 406, and a pyramid generation circuit 408. The multi-mode demosaicing circuit 404 is configured to receive raw image data from the image sensor 202, and to demosaic the received image data to output full-color image data. For example, as discussed above, demosaicing may include operations for converting and/or interpolating missing color samples from the raw image data (e.g., Bayer image data) to output image data into a full-color domain (e.g., RGB image data), and may include low pass directional filtering on interpolated samples to obtain full-color pixels.

The multi-mode demosaicing circuit 404 is configured to receive raw image data in a plurality of different image formats. For example, in some embodiments, the image sensor 202 may be configured to operate in a first mode to capture raw image data in a first format, and a second mode

to capture raw image data in a second format. In some embodiments, the first format may correspond to Bayer image data 410, while the second format corresponds to Quad Bayer image data 412 (hereinafter also referred to as “Quadra image data”). In some embodiments, the multi-mode demosaicing circuit 404 receives image data in different raw image formats (e.g., Bayer image data 410 and Quadra image data 412) from different image sensors 202.

FIGS. 5A and 5B illustrate examples of raw image data in the Bayer image format and Quadra image format, in accordance with some embodiments. Each pixel in the raw image data corresponds to a particular color component: red (R), blue (B), or green (G). As illustrated in FIG. 5A, in Bayer raw image data, each row of Bayer image data contains alternating green pixels and non-green pixels, where the non-green pixels the rows alternate between red and blue. On the other hand, as illustrated in FIG. 5B, in Quadra raw image data, the captured pixels are grouped into 2x2 blocks, with each 2x2 block of pixels being of the same color.

In some embodiments, the image sensor 202 may be a 48 megapixel sensor configured to capture Quadra image data when zoomed in, and Bayer image data when zoomed out. For example, when zoomed in, a center region of pixels of the image sensor is used to capture image data as Quadra image data. However, when zoomed out, blocks of 2x2 pixels may be combined to form one pixel of Bayer image data, resulting in a wider view captured with larger, lower-resolution pixels. In some embodiments, the image sensor 202 may switch between Quadra and Bayer image modes depending upon zoom factor, an amount of luminance in a scene, and/or other factors.

The front-end scaler circuit 406 is configured to receive the full-color image data generated by the multi-mode demosaicing circuit 404 and perform additional post-processing operations on the received image data, such as removal of color aliasing artifacts near luminance edges, dot-removal to remove dot-artifacts produced by the demosaicing circuit, etc. In addition, the front-end scaler circuit 406 may scale the received image data based on the requirements of the subsequent operations to be performed on the image. In some embodiments, downscaling is performed to reduce an amount of computation required in the back-end of the pipeline, while upscaling may be performed to improve fusion quality when the raw pyramid is used in temporal filtering.

For example, in some embodiments the scaling circuit 406 may operate in a first non-scaling mode in which the front-end scaler circuit 406 performs chroma aliasing artifact suppression and/or dot-removal operations, but does not perform any scaling, or a scaling mode in which downsampling or upscaling with chroma suppression is performed following chroma suppression and/or dot-removal operations. In some embodiments, scaling is performed by converting received full-color image data to YCC image data and from 4:4:4 format to 4:2:2 format by smoothing and downsampling the chrominance to save line buffer space, followed by vertical resampling and horizontal resampling based on the desired scale to produce an output image. As such, in some embodiments, the non-scaling mode may also be referred to as 4:4:4 mode, while the scaling mode is referred to as 4:2:2 mode.

The pyramid generation circuit 408 is configured to generate a pyramid containing multiple octaves representing multiple scales of an input image. In some embodiments, the pyramid generation circuit 408 receives full-color image data generated by the front-end scaler circuit 406, and

15

generates at least two octaves corresponding to at least a full-resolution luminance image (Scale 0) and a scaled full-color image (Scale 1). In other embodiments, the pyramid generation circuit 408 may receive raw image data (e.g., Bayer image data 410 and/or Quadra image data 412) for which to generate a raw image pyramid, bypassing the multi-mode demosaicing circuit 404 and the front-end scaler circuit 406.

FIG. 6 is a block diagram of a more detailed view of the multi-mode demosaicing circuit 404 and the scaling circuit 406, in accordance with some embodiments. As illustrated in FIG. 6, the multi-mode demosaicing circuit 404 may comprise different circuitry for demosaicing image data received in different image formats, such as a Bayer demosaicing circuit 610 for demosaicing Bayer image data, and a Quadra demosaicing circuit 612 for demosaicing Quadra image data. The multi-mode demosaicing circuit 404 includes logic circuitry 619 (e.g., a demultiplexor) configured to receive raw image data, determine a format of the received image data, and transmit the received image data to an appropriate demosaicing circuit for demosaicing (e.g., Bayer image data to the Bayer demosaicing circuit 610, or Quadra image data to the Quadra demosaicing circuit 612) to output full-color image data 618.

The Bayer demosaicing circuit 610 and the Quadra demosaicing circuit 612 access a demosaic shared memory 614 for use as working memory when performing demosaicing operations on received image data. In some embodiments, the demosaic shared memory 614 includes a set of line buffers used by the Bayer demosaicing circuit 610 and the Quadra demosaicing circuit 612 for storing working data when performing linear interpolation, gradient calculations, gaussian filtering, weighted averaging, etc. In some embodiments, the multi-mode demosaicing circuit 404 is configured such that only one of the Bayer demosaicing circuit 610 and the Quadra demosaicing circuit 612 are operating at a given time. As such, the Bayer demosaicing circuit 610 and the Quadra demosaicing circuit 612 may each utilize the full capacity of the demosaic shared memory 614 during operation.

In addition, the Bayer demosaicing circuit 610 and the Quadra demosaicing circuit 612 may utilize shared arithmetic logic 616 when demosaicing image data. For example, in some embodiments, the shared arithmetic logic 616 includes circuitry for performing operations such as division of fixed numbers, interpolation functions, logarithm and/or exponential functions, algebraic functions, etc. used by both the Bayer demosaicing circuit 610 and the Quadra demosaicing circuit 612. For example, in some embodiments, the Bayer demosaicing circuit 610 and the Quadra demosaicing circuit 612 may both include a gradient filter and/or a weighted averaging circuit that make use of the shared arithmetic logic 616 for performing certain operations.

The front-end scaler circuit 406 is configured to receive full-color image data 618 generated by the multi-mode demosaicing circuit 404, and includes one or more scaling circuits configured to performing scaling operations based on different image formats in which the image data was originally received by the multi-mode demosaicing circuit 404. For example, as illustrated in FIG. 6, the front-end scaler circuit 406 includes a Bayer scaling circuit 620 configured to perform scaling on image data that was originally received in Bayer format, and a Quadra scaling circuit 622 configured to perform scaling on image data originally received in Quadra format. In other words, even though the image data 618 output by the multi-mode demosaicing circuit 404 and received by the front-end scaler

16

circuit 406 may be full-color RGB image data regardless of whether the multi-mode demosaicing circuit 404 originally received the image data as Bayer image data or Quadra image data, the front-end scaler circuit 406 may be configured to perform different operations on the image data 618 depending on whether it was originally Bayer or Quadra image data.

In some embodiments, the front-end scaler circuit 406 performs different post-processing operations on the image data 618 based on its original raw image format, due to differences in how image data of different formats may be demosaiced. For example, because the pixels of Quadra image data are arranged in 2x2 blocks, pixels of the same color may have neighboring pixels of different colors, causing interpolation issues that may lead to poor diagonal sampling. For example, as illustrated in the kernel 806 in FIG. 8, the red pixel RI is located adjacent to green pixels along the right upward diagonal, allowing for interpolation based on nearby green pixels in the right upward diagonal direction to be performed easily. On the other hand, the red pixel R2 is not positioned near green pixels along the right upward diagonal direction, which may contribute to jaggy diagonal edges when interpolated. In some embodiments, the Quadra scaling circuit 622 performs additional post-processing operations, e.g., additional dot-fix and chroma suppression operations, in comparison to the Bayer scaling circuit 620, e.g., to refine diagonal lines to compensate for poor diagonal sampling when interpolating Quadra raw image data, whereas such operations may not be necessary when post-processing Bayer raw image data, in which the position of green pixels are equally distributed relative to each non-green pixel of the raw image data.

The multi-mode demosaicing circuit 404 and the front-end scaler circuit 406 access a shared memory 602 for use as working memory when performing demosaicing, post-processing, and scaling operations. In some embodiments, the shared memory 602 contains multiple banks of line buffers usable by both the multi-mode demosaicing circuit 404 and the front-end scaler circuit 406. In some embodiments, the shared memory 602 contains a first bank of line buffers 630 and a second bank of line buffers 632, where the second bank of line buffers 632 contains a larger number of line buffers in comparison to the first bank 630.

FIG. 6 illustrates access by the Bayer demosaicing circuit 610, Quadra demosaicing circuit 612, Bayer scaling circuit 620, and Quadra scaling circuit 622 to the line buffer banks 630 and 632 (as well as access by the Bayer demosaicing circuit 610 and Quadra demosaicing circuit 612 to the demosaic shared memory 614 and shared arithmetic logic 616) during different modes of operation by the demosaic processing circuit. For example, FIG. 6 illustrates access to the line buffer banks 630/632 during a first mode of processing of raw Bayer image data using dotted lines, while access to the line buffer banks 630/632 during a second of processing raw Quadra image data using solid lines.

Because the Quadra demosaicing circuit 612 may utilize additional working memory when demosaicing Quadra image data in comparison to the Bayer demosaicing circuit 610 when demosaicing Bayer image data, the Quadra demosaicing circuit 612 may access the second bank of line buffers 632 when performing demosaicing (as indicated by a solid line between Quadra demosaicing circuit 612 and the second line buffer bank 632), while the Bayer demosaicing circuit 610 accesses the smaller first bank of line buffers 630 when performing demosaicing (as indicated by a dotted line between Bayer demosaicing circuit 610 and the first line buffer bank 630, via "A"). On the other hand, the Quadra

scaling circuit **622** may access the first bank of line buffers **630** when performing post-processing and scaling on image data generated by the Quadra demosaicing circuit **612** (via “B”), while the Bayer scaling circuit **620** accesses the second bank of line buffers **632** when performing post-processing and scaling on image data generated by the Bayer demosaicing circuit **610**. In other words, the Quadra scaling circuit **622** may have access to smaller amount of memory of the shared memory **602** in comparison the Bayer scaling circuit **620**.

In some embodiments, the Quadra scaling circuit **622** performs its post-processing and scaling operations with reduced memory consumption in comparison to the Bayer scaling circuit **620** by discarding a portion of its received image data (e.g., certain lines of image data), and later reconstructing the discarded image data (e.g., reconstructing the discarded lines through interpolation).

Although FIG. 6 illustrates the scaler circuit **406** as having separate scaling circuits for Bayer and Quadra, it is understood that in some embodiments, the same scaling circuit or portions thereof may be used for scaling both Bayer and Quadra image data. In some embodiments, the scaler circuit **406** may utilize different settings when scaling Quadra image data compared to when scaling Bayer image data, such as utilizing different filter kernels, performing different functionality (e.g., to reduce memory consumption when processing Quadra image data in comparison to Bayer image data, as discussed above), etc.

By utilizing shared memory **602** used by both the multi-mode demosaicing circuit **404** and the front-end scaler circuit **406**, Quadra image processing functionality may be added to legacy Bayer image processing circuits, without significantly increasing an amount of area on chip needed for memory. For example, because Quadra demosaicing may utilize larger filters in comparison to Bayer demosaicing, thus requiring additional memory, operations of the front-end scaler circuit are modified (e.g., by discarding a portion of the image data so that a smaller amount of image data needs to be processed) to reduce memory usage, allowing for the combined circuit to utilize a similar amount of memory when processing Quadra image data in comparison to when processing Bayer image data, so that the same memory (e.g., shared memory **602**) is efficiently utilized in either image processing mode.

Quad Bayer Demosaicing

FIG. 7 illustrates a block diagram of a Quadra demosaicing circuit, in accordance with some embodiments. The Quadra demosaicing circuit **700** illustrated in FIG. 7 may correspond to the Quadra demosaicing circuit **612** illustrated in FIG. 6. The Quadra demosaicing circuit **700** is configured to demosaic received raw Quadra image data **412** by first producing a complete green color image, and, based on the produced green image, generate red and blue color images to produce full-color image data **618** (e.g., RGB image data).

The Quadra demosaicing circuit **700** includes a Horizontal/Vertical (HV) processing circuit **702** and a diagonal processing circuit **704**, each containing a respective linear interpolation circuit (e.g., linear interpolation HV circuit **712** and linear interpolation diagonal circuit **722**) and a respective directional Gaussian filter circuit (e.g., Gaussian filter circuits **714** and **724**) to perform linear interpolation and filtering on the raw Quadra image data in the horizontal/vertical and diagonal directions respectively, a respective gradient calculation circuit (e.g., gradient HV circuit **710** and gradient diagonal circuit **720**) to determine directional gradients based on the raw Quadra image data, and respective aggregation circuits (e.g., weighted averaging circuits

716 and **726**) to output an aggregation of the generated linear interpolation data based on the determined gradient data. The output of the HV processing circuit **702** and the diagonal processing circuit **704** is aggregated by a weighted average circuit **730** to generate a complete green image. The guided filter **732** uses the complete green image and the raw Quadra image data to generate complete red and blue images, which are combined with the complete green image to yield a full-color RGB image. These components will be discussed in greater detail below.

Although FIG. 7 illustrates a Quadra demosaicing circuit **700** with separate HV and diagonal processing circuits **702** and **704**, it is understood that in some embodiments, the HV and diagonal processing circuits may share circuit elements. For example, in some embodiments, a single interpolation circuit may be used to perform interpolation in HV and diagonal directions.

Quad Bayer Demosaicing: Linear Interpolation

The linear interpolation HV circuit **702** and linear interpolation diagonal circuit **712** (collectively, linear interpolation circuits **702/712**) of the Quadra demosaicing circuit **700** are each configured to perform linear interpolation on the raw Quadra input data in specified directions, to generate a green value for each non-green pixel of the image data. For example, to perform interpolation in the horizontal direction, the linear interpolation circuit HV circuit **702** determines, for each non-green pixel, a green value based on the values of nearby pixels along the horizontal direction (e.g., on the same row). As such, the linear interpolation circuit is configured to output an image of green pixels, including the original green pixels of the image data, and interpolated green pixels replacing the red and blue pixels of the original image data.

FIG. 8 illustrates examples of interpolation kernels that may be used to perform linear interpolation, in accordance with some embodiments. The example kernels illustrated in FIG. 8 may be used for interpolating a red pixel within the received raw Quadra image data, e.g., the red pixel **502** illustrated in FIG. 5. Kernels **802** and **804** correspond to a vertical interpolation kernel and a horizontal interpolation kernel, respectively, used by the linear interpolation HV circuit **704** for interpolating a red pixel **R0**. For example, a green value for the pixel **R0** may be determined based on the values of nearby green pixels **G1** through **G4**, e.g., a weighted average of **G1** through **G4**. It is understood that while FIG. 8 illustrates certain kernels that may be used for directional interpolation, in other embodiments, different kernels may be used. For example, in some embodiments, a green value for the pixel **R0** may be determined based on a weighted average of **G1**, **G2**, and **G3**, e.g., $(G1+2*G2+G3)/4$.

Kernel **806** illustrated in FIG. 8 corresponds to a diagonal interpolation kernel that may be used by the linear interpolation diagonal circuit **722**, for interpolating the red pixels **R1** through **R4**. For example, when interpolating along an upward right diagonal, the green value of the pixel **R1** may be based upon (e.g., an average of) pixels **G2** and **G7**, while pixel **R2** is interpolated based on green pixels **G2** and **G3**, pixel **R3** is interpolated based on green pixels **G6** and **G7**, and pixel **R4** is interpolated based on green pixels **G3** and **G6**. On the other hand, when interpolating along a downward right diagonal, pixel **R1** is interpolated based on **G1** and **G8**, **R2** based on **G1** and **G4**, **R3** based on **G5** and **G8**, and **R4** based on **G5** and **G4**.

In some embodiments, the value of each interpolated pixel is calculated based on a low pass value (LPV) and a high pass value (HPV). The LPV is calculated based on the

interpolation of the neighboring green pixels along the specified direction of interpolation, as discussed above, while the HPV is based upon the original non-green value (i.e., red or blue) of interpolated pixel. In some embodiments, the HPV may also be referred to as the residual. For example, the green value of the pixel R0 (e.g., LPV) may further be modified based upon the values of nearby red pixels along the specified direction (e.g., using the HPV or residual, which is determined based on R0, as well as nearby same-color pixels R1 and R2). For example, in some embodiments, the residual value may be determined as a function of $2 \cdot R0 - R1 - R2$. In some embodiments, a value of the residual may be clipped based upon one or more specified threshold values.

In some embodiments, mixing information from red or blue color information (e.g., the residual) is done to enhance the resolution of the green channel. However, due to sparse sampling associated with the Quadra image format, the green pixels and red/blue pixels may not represent the same data. As such, in some embodiments, the amount of the residual that is used to modify the interpolated green value for a pixel is based upon a calculated residual weight that is based on a statistical correlation between the nearby green and red or blue pixels. In some embodiments, the green pixels used for calculating the statistical correlation may be the same pixels used to determine the interpolated LPV values for the pixels, while the red or blue pixels may include red/blue pixels near the green pixels, based on the direction of interpolation.

FIG. 9 illustrates example diagrams of different amount of residual values added when interpolating image data, in accordance with some embodiments. FIG. 9 illustrates three diagrams 902, 904, and 906, each showing a different distribution of green and red pixel values surrounding a center pixel to be interpolated, where the green and red pixels exhibit different levels of correlation in each diagram. For example, diagram 902 illustrates an example where there is no correlation between the green and red signal. As such, when interpolating the center pixel, the residual weight is 0, resulting in no residual being added to the interpolated LPV value. In diagram 904, the green and red values partially correlate. As such, a percentage of the residual (e.g., ~38%) is used to modify the interpolated LPV value, based on the amount of correlation. On the other hand, if the green and red values fully correlate, e.g., as shown in diagram 906, then the full residual is added to the interpolated LPV value to generate the interpolated value for the pixel. In some embodiments, a value of the residual may be further modified based on one or more manually configured parameters. In some embodiments, the interpolated green values are modified based on residual values for all interpolation directions. In other embodiments, modifying the interpolated green values based on a residual value is performed for only certain directions (e.g., for only horizontal and vertical directions, but not diagonal directions).

In some embodiments, prior to interpolation, the linear interpolation circuit 712/722 analyzes the pixels of the raw Quadra image data to exclude certain pixels from interpolation. For example, in some cases an image may contain pixels, referred to as "bright dots," having high saturation values relative to surrounding pixels, such as in an image of sunlight shining through foliage. In other cases, bright dots may correspond to missed defective pixels. If interpolated normally, the saturation of the pixel may be spread around to surrounding pixels, creating visual artifacts, such as an undesirable cross or box, around the bright dot. As such, in some embodiments, when interpolating a value of a given

pixel, a nearby pixel may be excluded from interpolation if the value of the nearby pixel exceeds a threshold amount (e.g., excluded from being included in the interpolation of the neighboring green pixels, or from the determination of the residual value). In some embodiments, the threshold is defined based on a signal value of the pixel being interpolated. For example, in some embodiments the user may define a curve mapping threshold levels to different signal levels, where pixels above the threshold are excluded from the interpolation. In some embodiments, the signal level of the pixel being interpolated is determined based on an analysis of a neighborhood surrounding pixels (e.g., a 3×3 neighborhood of pixels), e.g., as a minimum pixel value, maximum pixel value, average pixel value, or some combination thereof of the neighborhood of surrounding pixels. The determined signal value is mapped to a threshold value for excluding bright pixels. In some embodiments, the threshold value is determined per color channel. In some embodiments, the threshold value may be set as a difference over an intensity value of the pixel being interpolated, instead of an absolute intensity level.

Quad Bayer Demosaicing: Directional Gaussian Filtering

FIG. 10 illustrates a block diagram of a Gaussian filter used in the Quadra demosaicing circuit, in accordance with some embodiments. The Gaussian filter illustrated in FIG. 10 may correspond to either the Gaussian filter 714 in the HV processing circuit 702, or the Gaussian filter 724 in the diagonal processing circuit 704 illustrated in FIG. 7, and includes a directional Gaussian filter 1002, a dot fix circuit 1004, and an orthogonal low-pass filter (LPF) 1006.

The directional Gaussian filter 1002 is configured to smooth and reduce noise of the interpolated green pixels generated by the linear interpolation circuit 702 or 712 by applying a directional Gaussian kernel on the received pixels. The dot fix circuit 1004 is configured to perform dot fixing operations on received image data. For example, in some embodiments, the dot fix circuit 1004 is configured to remove dots by analyzing the values of a center pixel and two or more neighboring pixels along a specified direction (e.g., the same direction in which interpolation was performed) to determine a minimum, maximum and median pixel value. If a difference between the minimum and maximum exceeds a dot fix threshold value and the center pixel has either the minimum or maximum value, then the dot fix circuit 1004 sets the value of the center pixel to the determined median pixel value. In some embodiments, the directional Gaussian filter 1002 may be bypassed (e.g., by setting a strength of the directional Gaussian filter 1002 to 0), such that the interpolated green pixels generated by the linear interpolation circuit 702 or 712 is received directly by the dot fix circuit 1004.

The orthogonal LPF 1006 is configured receive interpolated image data, and to apply a directional low-pass filter that is orthogonal to a direction in which the image data was interpolated, in order to reduce fixed pattern noise that may be generated by the interpolation. For example, when interpolating Quadra image data, noise present in certain pixels may be elongated by the interpolation, creating lines in the image in the direction of the interpolation, e.g., vertical interpolation would elongate the noise in a vertical direction, creating vertical lines. This may be especially noticeable in areas of images depicting flat surfaces or uniform colors, such as a patch of sky or a solid-color shirt. In addition, some image sensors configured to capture Quadra image data may experience gain and shift issues (e.g., electric signal cross-talk), where neighboring 2×2 blocks of green pixels may capture different raw image data even when the underlying

color being captured should be the same. These differences may result in fixed pattern noise when the raw image data is interpolated.

In order to reduce fixed pattern noise, the orthogonal LPF **1006** applies a low-pass filter kernel to the interpolated image data in a direction that is orthogonal to the direction in which the image data was interpolated. FIG. **11** illustrates examples of different kernels that may be used by orthogonal LPF, in accordance with some embodiments. For example, FIG. **11** illustrates a horizontal filter kernel **1102**, a vertical filter kernel **1104**, a first diagonal filter kernel **1106** (corresponding to the downwards right diagonal), and a second diagonal filter kernel **1108** (corresponding to the upwards right diagonal). When a horizontal LPF kernel, such as horizontal kernel **1102**, is applied to image data that was vertically interpolated, the vertical lines caused by fixed pattern noise are reduced, due to blending with adjacent pixels in a horizontal direction. Thus, by applying an orthogonal LPF, false patterns caused by fixed pattern noise may be removed.

While FIG. **11** illustrates examples of directional low-pass filters that may be used by the orthogonal LPF **1006**, in some embodiments, different filter kernels may be used to apply different levels of noise removal. In some embodiments, the strength of the noise removal is selected as a function of gradient strength (e.g., as determined by the gradient HV circuit **710** or gradient diagonal circuit **720**). By varying noise removal based on gradient, portions of the image corresponding to real textures may be avoided (e.g., with little or no noise removal applied), while applying noise removal to portions of the image corresponding to flat areas. Quad Bayer Demosaicing: Gradient Calculation

The gradient HV circuit **710** and gradient diagonal circuit **720** (collectively, gradient calculation circuits) are configured to calculate gradients within a received image, which are used to determine directional gradient weight values for combining image data interpolated along different directions (e.g., combining vertically-interpolated and horizontally-interpolated image data, image data interpolated in different diagonal directions).

In some embodiments, a directional gradient weight for a pixel is determined by analyzing a window of surrounding pixels (e.g., a 7×7 window). In some embodiments, due to the pixels in Quadra image data being grouped into 2×2 blocks, gradient calculation for Quadra image data may use a larger window in comparison to that used for determining gradient values for other image formats (e.g., Bayer images).

FIG. **12** illustrates examples of filter windows that may be used by the gradient calculation circuits, in accordance with some embodiments. The filter windows illustrated in FIG. **12** are 7×7 filter windows used for calculating gradients in the horizontal direction, although it is understood that similar filter windows may be used to calculate gradients for other directions. The numbers within the pixels in the illustrated filter windows correspond to filter coefficients, while pixels with no numbers represent pixels that are not considered in the gradient calculation using that particular filter window. The pixels of each row are aggregated (e.g., summed) based on their respective filter coefficient values, and the values for each row are aggregated to determine a final gradient value for the center pixel.

Filter window **1202** is a second-order filter for finding a gradient in a same color. For each horizontal row of the filter window **1202**, pixels of the same color channel are analyzed, and the results of each row are aggregated to generate an estimated horizontal gradient weight value W_h for the center pixel of the window (e.g., an absolute value of a weighted

sum of the indicated pixels in each row). Although the filter window **1202** only shows using pixels of certain columns in each row of the filter window (e.g., first, fourth, and fifth columns corresponds to pixels of a same color in each row) for finding the second-order same-color gradient, it is understood that in other embodiments, the filter window **1202** may also take into account color values of additional columns as well. For example, in some embodiments, a result value for each row may be based on a first aggregation of pixels of a first color (e.g., first, fourth, and fifth columns as illustrated in FIG. **12**) and a second aggregation of pixels of a second color (e.g., second, third, fifth, and sixth column pixels, having weight values of 1, 2, -2, -1, respectively).

However, because pixels in Quadra image data are grouped into 2×2 blocks, there is a large gap between pixels of the same color channel. In addition, as shown in FIG. **12**, the analyzed pixels in each row are not symmetrical. These issues caused by the format of Quadra image data may lead to inaccuracies in the determined gradient weight value W_h if only the second-order filter window **1202** is used.

In some embodiments, in order to improve an accuracy of the determined gradient weight value W_h , additional filter windows are applied to the Quadra image data, the results of which are combined to yield a more accurate gradient value for the center pixel. FIG. **12** illustrates a second filter window **1204** that determines a cross-color gradient value W_c . To determine the cross-color gradient, each row is divided into a plurality of cross-color pairs, each pair including a green pixel and a non-green pixel. For example, the cross-color pair **1208** includes a left red pixel, and a right green pixel. A value for each row is calculated as an aggregation (e.g., sum) of the absolute values of the differences between the pixels of each cross-color pair (e.g., for the cross-color **1208**, an absolute value of the right green pixel subtracted by the left red pixel). The values of the rows are then aggregated to determine the cross-color gradient W_c for the center pixel.

In addition, a third filter window **1206** corresponding to a first-order same-color filter may also be used to determine a gradient error E_h with each color. As shown in FIG. **12**, each row of the third filter window **1206** is divided into a plurality of same-color pairs. For example, the same-color pair **1210** includes a pair of adjacent green pixels. A value for each row is calculated as an aggregation (e.g., sum) of the absolute values of the differences between the pixels of each same-color pair. The values of the rows are then aggregated to determine the gradient error value E_h .

FIG. **13** illustrates an example of how the different filter windows for determining same-color gradient, cross-color gradient, and gradient error are combined to generate a final gradient value, in accordance with some embodiments. The original image sample **1302** is an example image of a Siemens star, and contains regions where strong horizontal gradients are present (e.g., region **1302a**), and other regions (e.g., region **1302b**) with little to no horizontal gradients.

The green gradient image **1304** illustrates a result of applying the first filter window **1202** on the original image **1302** to determine horizontal a second-order same-color (e.g., green) gradient weight W_h . In the illustrated image, white areas represent areas with detected gradients, while black areas correspond to areas with little or no detected gradient. As shown in FIG. **13**, the green gradient image **1304** is able to detect gradients in region **1302a**, but is not able to detect gradients in areas closer to the center of the Siemens star, such as areas where the image content nears Nyquist frequency, as shown in the region **1304a**, due to the

large gaps between analyzed pixels of the same-color gradient filter window **1202** used to determine the gradient.

In order to calculate more accurate gradient values, cross-color and gradient error filters (e.g., corresponding to filter windows **1204** and **1206**) are used to supplement the gradient values determined using the first filter window **1202**. For example, when the cross-color gradient filter window **1204** is applied to the original image **1302**, cross-color gradient values Ch as shown in the cross-color gradient image **1306** are obtained. As shown in the cross-color gradient image **1306**, application of the cross-color gradient filter window **1204** is able to yield more accurate gradient information near the center of the Siemens star, but also results in additional noise in other areas of the image.

In addition, the gradient error values Eh as shown in the same-color gradient image **1308** are obtained when the first-order same-color filter window **1206** is applied on the original image **1302**. The gradient error values Eh may be used to guide how much of the cross-color gradient Ch is used to modify the original same-color gradient Wh .

In some embodiments, the cross-color gradient Ch is normalized by using the cross-color gradient values calculated along another direction. For example, each gradient calculation circuit is configured to determine gradient values in at least two directions (e.g., the gradient HV circuit **710** configured to calculate horizontal and vertical gradients, and gradient diagonal circuit **720** configured to calculate diagonal gradients). As such, in the case of horizontal gradient calculation as discussed above, the horizontal cross-color gradient Ch is normalized using the vertical cross-color gradient Cv (not shown in FIG. **13**), the result of which is modified by the gradient error Eh and used to supplement the original second-order same-color gradient Wh . For example, in some embodiments, the final horizontal gradient value Wh is calculated as $Wh + Eh * (Ch / (Ch + Cv))$. The final combined gradient image **1310** illustrates the resulting gradient that is obtained when this aggregation is performed, showing more accurate gradient values near the center of the image, while exhibiting less noise in other areas of the image. As such, by combining the gradient data obtained using the second-order filter with first-order and cross-color filter data, gradient detection issues stemming from the unique image format of Quadra relative to Bayer, such as greater distances between same-color pixels and non-symmetric filter kernels, are reduced, yielding more accurate gradient values compared to what would be been possible with only the original second-order filter.

Although FIGS. **12** and **13** illustrate example methods of gradient determination in the context of horizontal gradients, it is understood that the similar techniques may be applied to determine gradients in other directions, such as vertical and/or diagonal gradients. In some embodiments, gradients corresponding to different directions may be determined differently. For example, in some embodiments, horizontal and vertical gradients are determined based upon a first-order gradient, second-order gradient, and cross-color gradient, such as that described above in relation to FIGS. **12** and **13**, while diagonal gradients are determined based upon first-order gradient but not second-order gradient (e.g., due to the different distribution of possible sample points along the diagonal direction in comparison to horizontal and vertical directions). For example, in some embodiments, diagonal gradients may be determined by first applying a filter mask to the raw image data to generate, for each pixel, a first-order gradient corresponding to an aggregation of differences between adjacent pixels along the diagonal, after which a filter (e.g., sum filter of a 3x3 pixel window) is

applied to determine a final diagonal gradient weight value (e.g., $Wd1$ and $Wd2$) for each pixel.

In some embodiments, gradient weight values are calculated only for a subset of pixels of the received image data (e.g., Quadra image data **412**). For example, in some embodiments, the gradient calculation circuits **710/720** are configured to calculate gradient weight values for pixels of certain color channels (e.g., only red and blue pixels, but not green pixels). Although FIGS. **12** and **13** illustrate filter windows centered on a green pixel, it is understood that gradients for red or blue pixels may be determined using the same techniques and similar filter windows centered on red or blue pixels.

Weighted average circuits **716** and **726** are configured to combine the interpolated image data generated by the linear interpolation circuits **712/722**, based upon the gradients generated by the gradient calculation circuits **710/720**. For example, the weight average circuit **716** combines horizontally interpolated green image data (Gh) and vertical interpolated green image data (Gv) generated by the linear interpolation HV circuit **702** (and filtered through the Gaussian filter **704**), based on horizontal and vertical gradient weight values Wh and Wv generated by the gradient calculation circuit **710**, to generate a weighted average Ghv . Similarly, the weight average circuit **726** combines green image data interpolated along two different diagonal directions ($Gd1$ and $Gd2$), based on gradient weight values along those diagonal directions ($Wd1$ and $Wd2$) to generate a weighted average Gd . In some embodiments where gradient values are determined for only a subset of pixels of the received image data, the gradient weight values used for determining a weighted average for pixel for which gradient values were not determined may use gradient values determined for a nearby pixel, or an aggregation of gradient values of two or more nearby pixels.

Weighted average circuit **730** is configured to combine the output image data of the weighted average circuits **716** and **726**, corresponding to a full green image processed in the horizontal and vertical directions Ghv and a full green image processed in both diagonal directions Gd , into a final full green image.

Quad Bayer Demosaicing: Guided Filter

The guided filter **732** produces red and blue color image planes using the previously generated green image plane as a similarity measure. In some embodiments, the guided filter **732** calculates, for each of red and blue, red and blue values for the pixels missing that color, and also replaces the original red and blue pixels. In some embodiments, the guided filter **732** functions as a low pass on the red and blue channels mixed with a high frequency of the green channel, where the amount of high frequency that is added from green to the red and blue values is based upon a covariance of the two signals.

FIG. **14** illustrates examples of guided filter windows that may be used to generate a red value for different types of pixels in a Quadra image, in accordance with some embodiments. It is understood that similar techniques to those described in relation to generating red pixel values may also be used to generate blue pixel values. The example windows illustrated in FIG. **14** are 5x5 windows. Window **1402** shows red pixels $R1$ through $R9$ used to determine a red value Rb for an originally blue pixel. Window **1404** shows red pixels $R1$ through $R6$ used to determine a red value Rg for an originally green pixel. Window **1406** shows red pixels $R1$ through $R4$ used to determine a replacement red value for an originally red pixel (including the original pixel itself). The values of the red pixels used to determine the red values are

obtained through the raw space image data (i.e., the original raw Quadra image data **412**) received by the Quadra demosaicing circuit **612**. In addition, the high frequency green color data from corresponding pixels may be added to the red values to determine the a final red value for the pixel, where the green color data is obtained from the green channel space data output by weighted average circuit **730**.

The guided filter **732** thus uses the original raw Quadra image data and the full green image to generate full red and blue images. The full green, red, and blue images are combined to form a full-color (e.g., RGB) image as the output of the Quadra demosaicing circuit **612**.

Example Process Flows

FIG. **15** is a flowchart of a process for multi-mode demosaicing and scaling of image data, in accordance with some embodiments. The process described by FIG. **15** may be performed by a demosaic processing circuit, such as demosaic processing circuit **402** illustrated in FIG. **4**, that contains a demosaicing circuit and a scaling circuit.

The demosaic processing circuit receives **1502** image data from one or more image sensors. In some embodiments, the demosaic processing circuit may be configured to receive image data from a single image sensors configurable to capture image data in different formats (e.g., Bayer image data when zoomed out above a certain level, or Quadra image data when zoomed in above a certain level), or from multiple different image sensors. In some embodiments, the demosaic processing circuit receives the image data from the one or more image sensors directly, while in other embodiments, the demosaic processing circuit receives the image data following one or more raw processing stage operations (e.g., black level compensation, highlight recovery and defective pixel correction).

The demosaic processing circuit determines **1504** a format of the received image data. For example, the demosaic processing circuit may determine whether the received image data is in a raw Bayer image format or a raw Quadra image format.

The demosaic processing circuit, based on the determination of the image format of the received image data (e.g., Bayer image format or Quadra image format), configures **1506** access to a shared memory by the demosaicing circuit and the scaling circuit. In some embodiments, the shared memory contains a plurality of line buffers, where, based upon whether the received image data is Bayer or Quadra image data, a number of line buffers within the shared memory accessible to each of the demosaicing circuit and the scaling circuit is different. For example, in some embodiments, demosaicing of Quadra data may utilize larger filter kernels in comparison to demosaicing Bayer image data. As such, the demosaicing circuit may access a larger number of line buffers in the shared memory when the received image data is Quadra image data, for use in demosaicing Quadra image data. In addition, a number of line buffers utilized by the scaling circuit may be reduced to compensate for the additional line buffers accessed by the demosaicing circuit.

The demosaic processing circuit demosaics **1508** the received image data using Bayer demosaicing logic or Quadra demosaicing logic, based on type of image data. For example, in some embodiments, the demosaicing circuit includes a Bayer demosaicing circuit and a Quadra demosaicing circuit, configured to receive image data in a raw Bayer format and in a raw Quadra format, respectively, and to demosaic the image data to output full-color image data (e.g., RGB image data). In some embodiments, the Bayer demosaicing circuit and Quadra demosaicing circuit use different filters and interpolation kernels for demosaicing

image data, but may share logic for performing certain operations, e.g., arithmetic operations.

The demosaic processing circuit post-processes and scales **1510** the demosaiced image data at scaling circuit, where operations performed and memory availability are based on original type of the received image data. For example, in some embodiments, the scaling circuit may perform different post-processing operations on received image data that was originally demosaiced from Quadra image data compared to Bayer image data, in order to remove artifacts that may be introduced into the image by the Quadra demosaicing process that may not be present in image data demosaiced from Bayer image data. In addition, the scaling circuit may be configured to, for post-processing and/or scaling operations on image data demosaiced from Quadra image data, perform different operations that utilize fewer line buffers in comparison to if the image data was demosaiced from Bayer image data, in order to compensate for additional line buffers used by the demosaicing circuit when demosaicing Quadra image data. As such, the demosaic processing circuit may be configured to perform demosaicing and scaling of either Bayer image data or Quadra image data, without increasing an amount of memory needed relative to previous circuits which only performed demosaicing and scaling on Bayer image data.

FIG. **16** is a flowchart of a process for demosaicing Quadra image data, in accordance with some embodiments. The process of FIG. **16** may be performed by a demosaicing circuit, such as the Quadra demosaicing circuit **612** illustrated in FIG. **6**.

The demosaicing circuit receives **1602** Quadra image data from one or more image sensors. The demosaicing circuit demosaics the received image data may performing linear interpolation of the image data along a plurality of directions to generate a plurality of interpolated green channels corresponding to the plurality of directions. In addition, the demosaicing circuit may, in parallel, identify gradient values of the received image data along the plurality of directions, for use in combining the interpolated image data associated with the different directions.

To interpolate the image data, the demosaicing circuit determines **1604**, for a non-green pixel of the image data, values of nearby green pixels and same-color pixels along each of a plurality of specified directions. The values of the nearby green pixels are aggregated to generate a base green value, while the values of the same-color pixels are aggregated to generate a residual value. The demosaicing circuit determines **1606** a residual weight based on a correlation between values of the green and non-green pixels along each direction. In some embodiments, the residual weight may indicate a percentage of the residual to be added to the base green value, where no residual is added if there is no correlation between the values of the green and same-color pixels, and the full amount of the residual is added if the values of the green and same-color pixels completely correlate. The demosaicing circuit interpolates **1608** a final green value for the pixel, based on the values of the nearby green pixels (e.g., the base green value) modified by the residual amount based on the residual weight.

In parallel to the interpolation, the demosaicing circuit determines gradient information for the image data. The demosaicing circuit generates **1610** a green channel gradient based on the Quadra image data along each of a plurality of directions. In some embodiments, the green channel gradient is generated using a second-order low pass filter. In addition, the demosaicing circuit generates **1612** a cross-color gradient based on the Quadra image data. In some embodiments,

27

cross-color gradient is based upon a sum of absolute values of the differences between pairs of adjacent pixels of different colors (e.g., a green pixel and a non-green pixel).

The demosaicing circuit determines **1614** an overall gradient based on the green channel gradient and the cross-color gradient. In some embodiments, the demosaicing circuit normalizes the determined cross-color gradient using a cross-color gradient determined along a different direction (e.g., vertical with horizontal, diagonal with opposite diagonal), and modifies the green channel gradient using the normalized cross-color gradient to obtain the final overall gradient values. In some embodiments, the demosaicing circuit further generates a same-color first-order gradient based upon a sum of absolute values of differences between adjacent same-color pixels, which is used to modify the normalized cross-color gradient value.

The demosaicing circuit combines **1616** the interpolated green channel data based on the gradient information to generate full-resolution green channel data. For example, in some embodiments, the demosaicing circuit generates a weighted average of the green channel data interpolated along different directions, using the determined gradient information as weights.

The demosaicing circuit combines **1618** the full-resolution green channel data with full-resolution red and blue channel data to generate demosaiced RGB image data. In some embodiments, the demosaicing circuit generates the red and blue image data by performing a low pass on red or blue channels modified by a high frequency of the full-resolution green channel data, where the amount of high frequency that is added from green to the red and blue values is based upon a covariance of the two signals. The generated full-resolution red and blue image data is combined with the full-resolution green image data to form the full-color demosaiced RGB image data.

While particular embodiments and applications have been illustrated and described, it is to be understood that the invention is not limited to the precise construction and components disclosed herein and that various modifications, changes and variations which will be apparent to those skilled in the art may be made in the arrangement, operation and details of the method and apparatus disclosed herein without departing from the spirit and scope of the present disclosure.

The invention claimed is:

1. An apparatus for demosaicing image data, comprising:
 - a logic circuit configured to receive image data from one or more image sensors and to determine whether the received image data is in a first image format or a second image format;
 - a first demosaicing circuit coupled to the logic circuit and a shared memory and configured to perform first demosaicing operations on the received image data to generate first full-color image data as an output responsive to the received image data being in the first image format; and
 - a second demosaicing circuit coupled to the logic circuit and the shared memory and configured to perform second demosaicing operations on the received image data to generate second full-color image data as the output responsive to the received image data being in the second image format, wherein the shared memory comprises a first number of buffers allocated to the first demosaicing circuit and a second number of buffers allocated to the second demosaicing circuit.

28

2. The apparatus of claim 1, wherein the first image format is a Bayer image format and the second image format is a Quadra image format.

3. The apparatus of claim 1, wherein the first number of buffers allocated to the first demosaicing circuit is smaller than the second number of buffers allocated to the second demosaicing circuit.

4. The apparatus of claim 1, wherein the first demosaicing operations or the second demosaicing operations comprise linear interpolation, gradient calculation, gaussian filtering, or weighted averaging.

5. The apparatus of claim 1, wherein the shared memory is coupled to the first demosaicing circuit and the second demosaicing circuit, and wherein the apparatus further comprises:

- a shared arithmetic logic circuit coupled to first demosaicing circuit and the second demosaicing circuit and configured to perform operations including division of fixed numbers, interpolation functions, logarithm functions, exponential functions, algebraic functions.

6. The apparatus of claim 5, wherein the first demosaicing circuit comprises a first gradient filter or a first weighted averaging circuit and the second demosaicing circuit comprises a second gradient filter or a second weighted averaging circuit, and wherein the first gradient filter, the first weighted averaging circuit, the second gradient filter, or the second weighted averaging circuit is configured to access the shared arithmetic logic circuit.

7. The apparatus of claim 1, wherein the logic circuit is configured to receive the image data from a single image sensor configured to capture the image data in the first image format and the second image format.

8. The apparatus of claim 1, wherein the logic circuit is configured to receive the image data from the one or more image sensors following one or more raw processing stage operations including black level compensation, highlight recovery, or defective pixel correction performed by a front-end pipeline stage circuit.

9. The apparatus of claim 1, further comprising:

- a front-end scaler circuit coupled to the first demosaicing circuit and the second demosaicing circuit and configured to:
 - receive the first full-color image data from the first demosaicing circuit or the second full-color image data from the second demosaicing circuit; and
 - perform post-processing operations on the received first or second full-color image data.

10. The apparatus of claim 9, wherein the post-processing operations comprise removal of color aliasing artifacts near luminance edges, dot-removal to remove dot-artifacts, downscaling or upscaling of the received first or second full-color image data.

11. The apparatus of claim 9, wherein the post-processing operations performed on the first full-color image data received from the first demosaicing circuit are different from the post-processing operations performed on the second full-color image data received from the second demosaicing circuit.

12. The apparatus of claim 9, wherein the front-end scaler circuit comprises:

- a first front-end scaler circuit configured to receive the first full-color image data from the first demosaicing circuit and to perform the post-processing operations on the received first full-color image data; and
- a second front-end scaler circuit configured to receive the second full-color image data from the second demo-

29

saicing circuit and to perform the post-processing operations on the received second full-color image data.

13. An electronic device comprising:

one or more image sensors configured to generate image data; and

a multi-mode demosaicing circuit coupled to the one or more image sensors, comprising:

a logic circuit configured to receive the image data and determine whether the received image data is in a first image format or a second image format;

a first demosaicing circuit coupled to the logic circuit and a shared memory and configured to perform first demosaicing operations on the image data to generate first full-color image data as an output responsive to the image data being in the first image format; and

a second demosaicing circuit coupled to the logic circuit and the shared memory and configured to perform second demosaicing operations on the image data to generate second full-color image data as the output responsive to the image data being in the second image format, wherein the shared memory comprises a first number of buffers allocated to the first demosaicing circuit and a second number of buffers allocated to the second demosaicing circuit.

14. The electronic device of claim **13**, wherein the first image format is a Bayer image format and the second image format is a Quadra image format.

15. The electronic device of claim **13**, wherein the shared memory is coupled to the first demosaicing circuit and the second demosaicing circuit, and wherein the multi-mode demosaicing circuit further comprises:

a shared arithmetic logic circuit coupled to first demosaicing circuit and the second demosaicing circuit and configured to perform operations including division of fixed numbers, interpolation functions, logarithm functions, exponential functions, algebraic functions, which are used by both the first demosaicing circuit and the second demosaicing circuit.

16. The electronic device of claim **13**, wherein the multi-mode demosaicing circuit further comprises:

a front-end scaler circuit coupled to the first demosaicing circuit and the second demosaicing circuit and configured to:

30

receive the first full-color image data from the first demosaicing circuit or the second full-color image data from the second demosaicing circuit; and perform post-processing operations on the received first or second full-color image data.

17. A method performed by a multi-mode demosaicing circuit,

comprising: receiving, by a logic circuit, image data from one or more image sensors;

determining whether the received image data is in a first image format or a second image format;

performing first demosaicing operations by a first demosaicing circuit on the image data to generate first full-color image data as an output responsive to the image data being in the first image format; and

performing second demosaicing operations by a second demosaicing circuit on the image data to generate second full-color image data as the output responsive to the image data being in the second image format,

wherein the first image format is a Bayer image format and the second image format is a Quadra image format, and wherein the image data comprises Bayer image data or Quad image data, and

wherein the first demosaicing operations or the second demosaicing operations comprise linear interpolation, gradient calculation, gaussian filtering, or weighted averaging.

18. The method of claim **17**, further comprising:

receiving the first full-color image data from the first demosaicing circuit or the second full-color image data from the second demosaicing circuit; and

performing post-processing operations on the received first or second full-color image data.

19. The method of claim **18**, wherein the post-processing operations comprise removal of color aliasing artifacts near luminance edges, dot-removal to remove dot-artifacts, and downscaling or upscaling of the received first or second full-color image data.

20. The method of claim **18**, wherein the post-processing operations performed on the first full-color image data received from the first demosaicing circuit are different from the post-processing operations performed on the second full-color image data received from the second demosaicing circuit.

* * * * *